

GreenCity

Автоматическое проектирование озеленения с учётом подземных коммуникаций и городской среды

Хакатон «Лидеры цифровой трансформации 2026», команда «Вороны»

Артемий Корнилов, Константин Жадько

Содержание

1. Документация решения
2. Алгоритм GreenPlan
3. Запуск и разработка
4. Технический обзор

GreenCity — документация решения

Сервис автоматического проектирования озеленения городских участков с учётом подземных коммуникаций и городской среды. Кейс хакатона «Лидеры цифровой трансформации 2026».

Команда «Вороны»: Артемий Корнилов, Константин Жадько.

Документ	Что в нём
этот документ	назначение и границы MVP, схема процесса, интерпретируемость, методы и библиотеки, проверка результата, ограничения, примеры
GREENPLAN_ALGORIT HM.md	алгоритм GreenPlan по шагам
TECHNICAL_OVERVIEW .md	техническое устройство: модули, API, парсер, ИИ-ассистент, экспорт, тесты
./README.md	запуск в Docker и локально, тесты, линтеры
CASE_BRIEF.md	описание кейса

1. Назначение и границы MVP

Назначение. Проектировщик загружает чертёж участка (DXF или папку DWG) и получает:

- план озеленения, который соблюдает нормативные отступы от зданий, дорог и подземных сетей;
- выходной DXF, в котором исходные слои не тронуты, а результат лежит на отдельных слоях;
- машиночитаемый файл объяснений со ссылкой на нормативный акт и пункт для каждой посадки;
- пояснительную записку с ведомостью элементов озеленения.

План можно поправить вручную на 3D-сцене или текстом через ИИ-ассистента.

Что входит в MVP:

Возможность	Где
Импорт DXF и папки DWG (конвертация LibreDWG), распознавание слоёв: граница, здания, сети с охранными зонами, дороги, дорожки, площадки, существующие деревья и МАФ	parser/ , backend/exchange/
3D-сцена с зонами ограничений и кольцами нормативных отступов, перетаскивание объектов с проверкой норм в реальном времени	frontend/
GreenPlan — озеленение по проектам-аналогам: стиль участка, приёмы зон, виды из ассортимента Москвы, расстановка с проверкой нормы в каждой точке, газон, благоустройство	backend/greenplan/ , алгоритм
Правка текстом: ИИ-ассистент возвращает операции, координаты считает планировщик	backend/text_editor/
Экспорт DXF поверх исходного чертежа: слои NEW_* и USER_*, XDATA с id посадки	backend/exchange/dxf_o verlay.py
Файл объяснений (JSON, CSV) и пояснительная записка (DOCX)	backend/greenplan/expl anations.py , document.py
Аккаунты и сохранённые проекты, мониторинг, Docker	backend/accounts/ , observability/

Что не входит в MVP:

- решения по существующим деревьям (сохранить, пересадить, вырубить) и компенсационный расчёт — в данных кейса нет перечётных ведомостей и оценок состояния;
- многолетники, цветники и луковичные в автоматической расстановке — GreenPlan сажает деревья, кустарники и газон; цветник можно поставить вручную или через ассистента;
- рельеф, инсоляция, почвы и дренаж — в DXF кейса этих данных нет;
- рабочая документация: посадочные ямы, приствольные лунки, разбивочные чертежи.

Пользователь — проектировщик благоустройства или эксперт, проверяющий проект. Интерфейс — веб-приложение, API описан в Swagger (в дев-режиме — <http://localhost:5173/docs>, в продакшене выключен).

2. Схема процесса

Процесс от входного DXF до выходного DXF и файла объяснений в нотации BPMN 2.0. Исходник схемы — [bpmn/greencity_process.bpmn](#), открывается в [Camunda Modeler](#) и на [demo.bpmn.io](#). SVG строит `tools/docs/build_bpmn.py`.

Подпроцесс GreenPlan раскрыт на отдельной схеме в [описании алгоритма](#).

Как читать схему:

- **Дорожки.** Слева — действия пользователя в редакторе, в центре — бэкэнд, справа — обращения к языковым моделям в Yandex AI Studio. Жёлтым выделены шаги с LLM. Оба необязательны: без ключа всё остальное работает.
- **Исходник.** Входной чертёж сохраняется на сервере по содержимому (sha256). Экспорт дописывает результат в копию именно этого файла, поэтому координаты, подписи, штриховки и нераспознанные слои исходника сохраняются.
- **Итог.** Экспорт DXF, файл объяснений и записка формируются по одной и той же сцене. id посадки один и тот же в сцене, в XDATA сущности DXF и в файле объяснений.

3. Как это устроено

```
DXF / DWG → parser (слои → сцена JSON) → backend FastAPI → frontend React + three.js
                                     |
                               GreenPlan · ИИ-ассистент · экспорт DXF · объяснения · записка
```

- **Сцена** (`backend/core/schemas.py`) — общий формат парсера, бэкенда и фронтенда:
 - граница участка;
 - зоны ограничений: тип, серьёзность (`forbidden`, `warning`, `allowed`), ширина охранной зоны;
 - объекты: здания, деревья, кустарники, МАФ;
 - газон;
 - привязка к исходному чертежу: масштаб, центр, id исходника.
- **Парсер** распознаёт слои по подстрокам в имени, на английском и русском (`parser/dxf_parsing/rules.py`). Линии сетей одного слоя объединяются и расширяются в охранные коридоры. Сети обрезаются по границе участка. Разрозненные отрезки зданий склеиваются в контуры. Если слоя границы нет, граница оценивается по содержимому.
- **Нормы** хранятся в коде (`backend/core/setback_norms.py`) и в машиночитаемых выписках из актов (`data/norms/`). Автотест сверяет их между собой.
- **Бэкэнд асинхронный.** Тяжёлые расчёты (разбор чертежей, GreenPlan, экспорт) идут в пуле потоков по одному на процесс через ограниченную очередь, пачка DWG — в отдельном процессе, запросы к LLM — асинхронным клиентом. Сервер отвечает на лёгкие запросы и во время расчёта.
- **Сеть и защита.** Наружу открыт только фронтенд (и Grafana): бэкэнд, базы и мониторинг доступны лишь внутри сети контейнеров, API проксирует фронтенд, CORS выключен. У тяжёлых эндпоинтов есть:
 - лимит частоты с одного IP (429);
 - лимит размера запроса (413);
 - ответ 503 с `Retry-After` при переполненной очереди.

Подробнее — [TECHNICAL_OVERVIEW.md](#).

4. Интерпретируемость: объяснение каждой посадки

Каждое решение сервиса можно проследить до входных данных и нормы. Механизм держится на трёх вещах.

1. Решения принимают правила, а не модель. Приём зоны выбирается голосованием проектов-аналогов, вид — фильтрами по актам, точка — геометрической проверкой отступов. Поэтому у каждого решения есть

конкретное основание, которое записывается в момент принятия решения, а не придумывается после:

Что решено	Основание	Где записано
Стиль участка	голосование к ближайших аналогов, ведущий аналог	<code>ZoneAssignment.site_style</code> , <code>lead_project</code>
Приём зоны	аналог, от которого взят приём, доля голосов	<code>pattern_id</code> , <code>source_project</code> , <code>confidence</code> ; без аналога — «типовое решение — проверить»
Вид растения	ассортимент Москвы для типа территории, 515-ПП, 369-ПП, коды ограничений	<code>species_basis</code>
Место посадки	отступ до каждого ограничения по таблице и правилам по породе	<code>setback_norms.setback_basis</code> → пункт НПА
Отказ в точке	правило, которое не выполнилось	журнал <code>RejectionLog</code>

2. Каждая норма знает свой пункт. `setback_basis(zone_type, kind, species)` возвращает не только расстояние, но и акт с пунктом, который его задаёт:

Ограничение	Акт и пункт
Табличные отступы от зданий, дорог, дорожек, сетей, опор освещения	СП 42.13330.2016, п. 9.6, табл. 9.1; ППМ 743-ПП, табл. 3.6.1
Широкая крона у здания (липа, клён, дуб, каштан, тополь — 10 м)	ППМ 743-ПП, табл. 3.6.1, прим. 3
Породы, чувствительные к прогреву, у теплосети	МГСН 1.02-02, п. 4.2.8
Колючие кустарники у дорожек и площадок	СП 82.13330.2016, п. 9.22
Экспертные оценки	помечены «экспертная оценка, не норма акта»
Зазор между соседними объектами	помечен «правило планировщика (не норма)»

3. Один id везде. У посадки один id в сцене, в XDATA GREENCITY сущности выходного DXF и в файле объяснений. Посадку, выбранную в CAD, можно найти в файле объяснений, и наоборот.

Файл объяснений — `POST /api/greenplan/explanations?format=json|csv`, кнопки «Объяснения посадок: JSON / CSV» на панели GreenPlan. Содержит:

- **plants** — каждая новая посадка, от GreenPlan и от пользователя:
 - id, вид, статус и происхождение;
 - слой DXF;
 - координаты сцены (`x`, `z`, метры от центра участка) и исходного чертежа (`dxf_x`, `dxf_y`);
 - вид зоны, приём, проект-аналог, стиль участка;
 - основание подбора вида;
 - до пяти ближайших ограничений: что это, фактическое расстояние, требуемый отступ, выполнено ли, акт и пункт;
 - итоговая фраза `reason`.
- **rejected** — точки, которые расстановка рассматривала, но не поставила по норме, с причиной, расстоянием, нормой и пунктом. Подробно — до 300 на участок, всего и по правилам — в `summary`.
- **restricted_zones** — каждое ограничение участка и нормативная полоса вокруг него для дерева и кустарника с пунктом НПА. В этих полосах расстановка точки не рассматривает вовсе.
- **site** — привязка координат: id источника, масштаб, центр участка в координатах чертежа.

Пример записи из `examples/12_natashinsky_proezd`:

```
{
  "id": "tree_building_border_001_001",
  "type": "tree",
  "species": "Черемуха Маака",
  "origin": "GreenPlan",
  "dxf_layer": "NEW_TREE",
  "dxf_x": 349.906, "dxf_y": 188.744,
  "zone_kind": "building_border",
  "pattern": "Кольцо кустарника вокруг здания",
  "source_project": "25_classical_building_ring_primer",
  "site_style": "пейзажный",
  "species_basis": "ассортимент для озеленения Москвы, категория «дворовые территории (тип участка не определён)»; в приоритете базовый ассортимент ППМ 515–ПП; без инвазивных видов ППМ 369–ПП",
  "constraints": [
    {"what": "кабель связи", "distance_m": 2.55, "required_m": 2.0, "ok": true,
      "norm": "СП 42.13330.2016, п. 9.6, табл. 9.1; ППМ 743–ПП, табл. 3.6.1"}
  ]
}
```

CSV (разделитель «;», UTF-8 с BOM — открывается в Excel) содержит одну строку на посадку с ближайшим ограничением, на отклонённую точку и на зону запрета.

Независимая проверка. Модуль нарушений заново проверяет все объекты сцены против всех зон, не используя код расстановки. Нарушения существующих объектов не скрываются: они идут в ответ, на панель и в записку.

Роль LLM. Текст-обоснование от YandexGPT — только пересказ готовых решений. Модели запрещено добавлять виды, площади и проекты, которых нет в сводке, а в записке этот текст идёт приложением с пометкой «текст ИИ — проверить». ИИ-ассистент возвращает операции («посадить вдоль дорожки», «убрать у площадки»), а координаты и проверку норм выполняет тот же планировщик.

5. Методы и библиотеки

Методы

Метод	Где применяется	Источник
Рассуждение по прецедентам (case-based reasoning): решение по похожим реализованным проектам	GreenPlan, поиск аналогов	A. Aamodt, E. Plaza. Case-Based Reasoning: Foundational Issues, Methodological Variations, and System Approaches . AI Communications, 1994
к ближайших соседей по косинусному сходству стандартизованных признаков	<code>pattern_retrieval.py</code>	C. Manning, P. Raghavan, H. Schütze. Introduction to Information Retrieval , гл. 6 и 14
Взвешенное голосование аналогов, раздача зон пропорционально весу	<code>pattern_assignment.py</code>	собственная реализация
Правила классификации типа территории	<code>site_characterization.py</code>	категории ассортимента озеленения Москвы (<code>data/norms/assortment-msk</code>)
Равномерный выбор точек: k-средних (алгоритм Ллойда)	<code>core/placement_geometry.pick_spread</code>	S. Lloyd. Least squares quantization in PCM . IEEE Trans. Inf. Theory, 1982

Метод	Где применяется	Источник
Poisson-disk разброс	генератор по сетке (<code>generation/natural_sampling.py</code>)	R. Bridson. Fast Poisson Disk Sampling in Arbitrary Dimensions. SIGGRAPH, 2007
Пространственный индекс STRtree	проверка нарушений, объяснения, генератор	S. Leutenegger, M. Lopez, J. Edgington. STR: A Simple and Efficient Algorithm for R-Tree Packing. ICDE, 1997
Минимальное остовное дерево (алгоритм Прима) для каркаса дорожек	<code>text_editor/courtyard_layout.py</code> , благоустройство GreenPlan	R. Prim. Shortest connection networks and some generalizations, 1957
Граф видимости и алгоритм Дейкстры для дорожки в обход зданий	<code>core/placement_geometry.shortest_path</code>	M. de Berg и др. Computational Geometry: Algorithms and Applications, гл. 15
Границы Тьюки для отсева выбросов, вогнутая оболочка для оценки границы участка	<code>parser/dxf_parsing/boundary_estimate.py</code>	J. Tukey. Exploratory Data Analysis, 1977; <code>shapely.concave_hull</code>
Точка, наиболее удалённая от края многоугольника (<code>polylabel</code>)	дизайн двора	<code>mapbox/polylabel</code> , <code>shapely.polylabel</code>
LLM как интерпретатор намерения: модель возвращает операции, код считает координаты	<code>text_editor/</code>	—

Библиотеки и сервисы

Бэкенд и обработка чертежей (Python 3.12)

Библиотека	Назначение
FastAPI, Starlette, Uvicorn	веб-сервис и Swagger
Pydantic	модели данных, валидация
ezdxf	чтение и запись DXF, импорт документов, XDATA
LibreDWG (<code>dwg2dxf</code>)	конвертация DWG в DXF
Shapely / GEOS	вся геометрия: буферы, объединения, пересечения, STRtree
NumPy	векторы признаков, стандартизация
<code>pyproj</code> , <code>pyshp</code>	конвертеры GeoJSON и SHP → DXF
<code>python-docx</code>	пояснительная записка
PyYAML	корпус решений <code>pattern_corpus.yaml</code>
<code>openai-python</code>	клиент OpenAI-совместимого API Yandex AI Studio
<code>PyJWT</code> , <code>bcrypt</code>	токены и пароли
<code>PyMongo</code> , <code>redis-py</code>	MongoDB (аккаунты, проекты), Redis (сессии, лимиты, кэш)
<code>prometheus-client</code> , <code>prometheus-fastapi-instrumentator</code>	метрики

Фронтенд (TypeScript)

Библиотека	Назначение
React, React Router	интерфейс

Библиотека	Назначение
three.js, React Three Fiber, drei	3D-сцена, инстансирование моделей
Vite	сборка
Tailwind CSS, Radix UI, Motion, Lucide	оформление и компоненты

Языковые модели — Yandex AI Studio, один ключ и каталог:

- ИИ-ассистент — Qwen3 235B A22B: строгий JSON по длинной инструкции;
- текст-обоснование — YandexGPT 5.1 Pro.

Инфраструктура:

Компонент	Назначение
Docker Compose	запуск
MongoDB, Redis	хранилища
nginx	статика в продакшене
Prometheus, Loki, Grafana	мониторинг

Разработка: pytest, Ruff, oxlint, TypeScript.

Данные и нормы:

- проекты и чертежи — набор данных кейса;
- ассортимент для озеленения Москвы — документ заказчика;
- нормативные акты: СП 42.13330.2016, СП 82.13330.2016, СП 59.13330.2020, ППМ 743-ПП, 515-ПП, 369-ПП, 616-ПП, МГСН 1.02-02, ГОСТ 21.508-2020 (выписки и пункты — data/norms/README.md);
- 3D-модели — паки, описанные в frontend/public/models/README.md.

6. Проверка выходного DXF

Выходной DXF — это исходный чертёж без изменений плюс слои результата:

Слой	Что на нём
NEW_TREE , NEW_BUSH	посадка GreenPlan: точка и окружность-маркер
NEW_LAWN	новый газон (заливка HATCH, клумбы — дырками)
NEW_PATHS , NEW_LAMP , NEW_BENCH , NEW_TRASH	благоустройство GreenPlan
USER_*	объекты, добавленные пользователем (вручную или ассистентом), и новое место сдвинутых исходных
USER_REMOVED	крестик на месте исходного объекта, который пользователь удалил или сдвинул; сам объект остаётся в своём слое
USER_ZONES	зоны, выделенные в редакторе

У каждой сущности результата есть XDATA приложения GREENCITY : id посадки, вид, происхождение («GreenPlan: приём, аналог» или «правка пользователя»).

Автоматическая проверка (любая ОС, в том числе Linux и МосТех.ОС)

Скрипт на Python и ezdxf, без CAD:

```
.venv/bin/python tools/check_export.py <исходный.dxf> <результат.dxf> [объяснения.json]
```

Что он проверяет:

1. Все сущности исходника на месте с теми же handle, типом, слоем и координатами.
2. Всё добавленное лежит только на слоях NEW_* и USER_* и несёт XDATA GREENCITY.
3. Если передан файл объяснений: каждая посадка из него есть в DXF на указанном слое, в координатах dxf_x, dxf_y с точностью 0,001.

Пример для участка из набора кейса:

```
$ .venv/bin/python tools/check_export.py
locations/12_natashinsky_proezd/12_natashinsky_proezd.dxf \
  examples/12_natashinsky_proezd/12_natashinsky_proezd_greenplan.dxf \
  examples/12_natashinsky_proezd/12_natashinsky_proezd_explanations.json
Исходных сущностей: 2626; пропало: 0; изменено: 0
Добавлено сущностей: 1233; вне слоёв NEW_*/USER_*: 0; без XDATA GREENCITY: 0
NEW_BUSH: 966
NEW_LAWN: 37
NEW_TREE: 230
Посадок в файле объяснений: 598; не найдено в DXF или не совпало: 0
ПРОВЕРКА ПРОЙДЕНА
```

Проверка в nanoCAD

1. **Открыть файл:** «Файл → Открыть», тип файла DXF, выбрать *_greenplan.dxf. Единицы и координаты исходника сохранены, чертёж встаёт на своё место.
2. **Слои:** открыть диспетчер слоёв (команда СЛОЙ / _LAYER). Кроме слоёв исходника должны быть NEW_TREE, NEW_BUSH, NEW_LAWN и, если были правки, USER_*.
3. **Исходник не тронут:** отключить все слои NEW_* и USER_*. На экране остаётся исходный чертёж: подписи, штриховки, блоки, сети — как в исходном файле. Можно открыть исходный DXF рядом и сравнить.
4. **Координаты:** выбрать точку посадки на NEW_TREE и посмотреть положение в панели «Свойства». Найти эту посадку в CSV объяснений по координатам (dxf_x, dxf_y) и сравнить вид, приём и норму.
5. **XDATA:** в версиях nanoCAD с LISP выполнить в командной строке (assoc -3 (entget (car (entsel))) ("GREENCITY"))) и выбрать посадку. Ответ содержит id, вид и происхождение — тот же id, что в файле объяснений. В AutoCAD то же показывает команда XDLIST (Express Tools).

Проверка на Linux и МосТех.ОС без nanoCAD

- **LibreCAD или QCAD** (свободные, есть в репозиториях дистрибутивов): открыть выходной DXF, в панели слоёв отключить NEW_* и USER_* и убедиться, что исходник цел. Выбрать точку посадки и сравнить координаты с файлом объяснений. XDATA эти программы не показывают — для неё используйте автоматическую проверку выше.
- **Сам GreenCity:** загрузить выходной DXF в редактор. Посадки читаются обратно без потери и удвоения, слои результата распознаются по подстрокам TREE, BUSH и LAWN. Это проверяет сквозной тест backend/tests/e2e/.

7. Ограничения, допущения, известные ошибки

Данные

- **Маленький и местами некорректный датасет.** В наборе кейса 14 документированных проектов. Этого мало для обучения модели, поэтому выбран подход по прецедентам. Реальные проекты описаны частично, обычно одним-двумя видами зон, поэтому для типовых случаев (двор с площадкой, бульвар, парковка, сквер над сетями) добавлены 20 синтетических эталонов с явной пометкой. Часть чертежей содержит ошибки, их исправления эвристические и могут ошибаться на незнакомых чертежах:
 - самопересекающиеся и вырожденные контуры (чинятся через `make_valid`);
 - тысячи обрывков одного слоя (объединяются);
 - сети подложки далеко за границей участка (обрезаются);
 - отсутствие слоя границы (граница оценивается по содержимому);
 - дубли объектов из разных файлов пачки DWG (удаляются).
- **Реальные участки входят в корпус.** Для участка из набора кейса ближайшим аналогом может оказаться он сам (например, `06_kamchatskaya_ulitsa`, `12_natashinsky_proezd` в примерах).
- **Решения по существующим деревьям не делались.** Сохранить, пересадить или вырубить, компенсационное озеленение — для этого нужны перечётные ведомости и оценка состояния деревьев, которых в данных нет. Существующие деревья учитываются как препятствия, и их нарушения норм показываются.
- **Нет данных о рельефе, почвах, инсоляции и подземных глубинах.** Отступ от сети считается в плане, без глубины заложения.

Допущения

- **Распознавание слоёв.** Слои распознаются по подстрокам имени (`parser/dxf_parsing/rules.py`). Слой с непривычным названием не распознаётся и остаётся в DXF, но не участвует в расчёте. Правила дополняются в одном файле.
- **Охранные зоны сетей.** Сети без ширины в чертеже получают ширину охранной зоны из правил слоя. Отступ посадки добавляется к ней, поэтому у коридоров сетей это даёт запас сверх нормы.
- **Эмпирические параметры.** Ширина полос у зданий, дорожек и края участка (3 м), шаги посадки, предел палитры (4 вида деревьев и 4 кустарников) подобраны по реальным проектам, а не взяты из норм.
- **Тип территории.** «Не определён» считается дворовой территорией при подборе видов.
- **Ассортимент 515-ПП.** Базовый ассортимент 515-ПП даёт приоритет для дворов, хотя акт обязателен только для территорий реновации.
- **Экспертные оценки.** Отдельные отступы по породе — экспертные оценки. Они помечены как таковые, а не выдаются за норму.

Известные ошибки и особенности

- **Экспорт без исходника.** Если исходника на сервере нет (проект сохранён до появления хранилища, файл удалён) или наложение на него не удалось (например, чертёж старого формата R12), экспорт собирается из сцены: в метрах от центра участка, только распознанные слои. Ответ помечен заголовком `X-GreenCity-Export: rebuilt`, редактор предупреждает.
- **Скорость экспорта DWG.** Экспорт поверх пачки DWG заново конвертирует её — на крупной пачке это около 20 секунд.
- **Лимиты частоты по IP.** Пользователи за одним адресом (офис за NAT) делят один лимит. Счётчики хранятся в Redis: если он недоступен, лимиты не действуют. Пороги настраиваются переменными окружения (README, «Сеть и защита API»).

- **Очередь расчётов на процесс.** Она своя у каждого процесса uvicorn, общей очереди на весь сервер нет. При пиковой нагрузке часть запросов получает 503 и должна повторить запрос позже.
- **Конвертация DWG.** LibreDWG не читает часть объектов (3D MESH, ACIS-тела) и некоторые версии формата. Такие файлы пачки пропускаются с предупреждением.
- **Округление в объяснениях.** Расстояния округляются до 0,01 м, поэтому у точки, отклонённой на самой границе нормы, расстояние может совпасть с нормой (например, 5,0 м при норме 5,0 м — на деле 4,996 м).
- **Газон на сцене.** Газон на 3D-сцене пересчитывается при следующем запуске GreenPlan. Записка и экспорт пересчитывают его сами по текущей сцене.
- **ИИ-ассистент.** Он может понять просьбу не так. На наборе из 29 реалистичных просьб он выполняет все, но это не гарантия для любой формулировки. Всё, что он предлагает, проходит проверку норм, невыполнимое отклоняется с причиной, последнюю правку можно отменить.
- **Объём посадки.** На очень крупных участках (десятки гектаров) посадка ограничена 10 000 объектов и прореживается равномерно.

8. Примеры входных и выходных DXF

В `examples/` — три участка из набора кейса, прогнанные через GreenPlan с параметрами по умолчанию. Входной DXF лежит в `locations/<участок>/`, результат собирает `tools/build_examples.py` через HTTP API, без LLM.

Участок	Стиль, ведущий аналог	Зон	Новых деревьев	Новых кустарников	Новый газон	Нарушений у новых	Отклонено точек
07_nizhnie_polya	пейзажный, 19_2ya_pryadilnaya	246	290	1253	670 м²	0	1065
06_kamchatskaya_ulitsa	регулярный, 06_kamchatskaya_ulitsa	114	80	660	12 748 м²	0	513
12_natashinsky_proezd	пейзажный, 12_natashinsky_proezd	44	115	483	2085 м²	0	269

«Отклонено точек» — кандидаты, не прошедшие проверку при расстановке: по норме и по зазору между соседними объектами. Разбивка по правилам — в `summary.rejected_by_rule` файла объяснений.

В папке каждого участка:

Файл	Что в нём
<code><участок>_greenplan.dxf</code>	выходной DXF: исходник и слои <code>NEW_*</code>
<code><участок>_explanations.json, .csv</code>	файл объяснений
<code><участок>_note.docx</code>	пояснительная записка (без приложения с текстом ИИ)
<code>summary.json</code>	сводка и ведомость по видам

Все три результата проходят `tools/check_export.py`.

9. Запуск

```
cp .env.example .env          # ключи Yandex AI Studio — необязательно
docker compose up -d --build
```

После запуска:

- редактор — `http://localhost:5173`;
- Swagger — `http://localhost:5173/docs` (через прокси фронтенда; бэкэнд наружу не опубликован);
- Grafana — `http://localhost:3001`.

Без ключа LLM всё работает, кроме ИИ-ассистента и текста-обоснования. Подробности, продакшен-режим и локальный запуск — в [README](#).

10. Лицензия

Код открыт только для просмотра: копирование, запуск, изменение и распространение без письменного согласия авторов запрещены. Исключение — организаторы и эксперты конкурса «Лидеры цифровой трансформации 2026» и ДПиООС: им разрешено собирать и запускать решение для оценки. Полный текст — [LICENSE](#).

Алгоритм GreenPlan

GreenPlan проектирует озеленение участка по аналогии с реализованными проектами на похожих участках. Он не придумывает решение с нуля и не обучает модель. Решения принимаются правилами и геометрией, поэтому их можно воспроизвести и проверить. Языковая модель не выбирает приёмы и не считает координаты. Она только пересказывает готовые решения связным текстом для записки, и этот шаг необязателен.

Вход: сцена участка, полученная разбором DXF или пачки DWG (граница, зоны ограничений, здания, существующие объекты), и параметры пользователя. Выход: сцена с новыми посадками, газоном и благоустройством, решения по каждой зоне с их происхождением, список нарушений, ведомость, журнал отклонённых точек.

Точка входа: `POST /api/greenplan/generate`, код в `backend/greenplan/pipeline.py`. Время работы — секунды, от 2 до 10 на обычном участке.

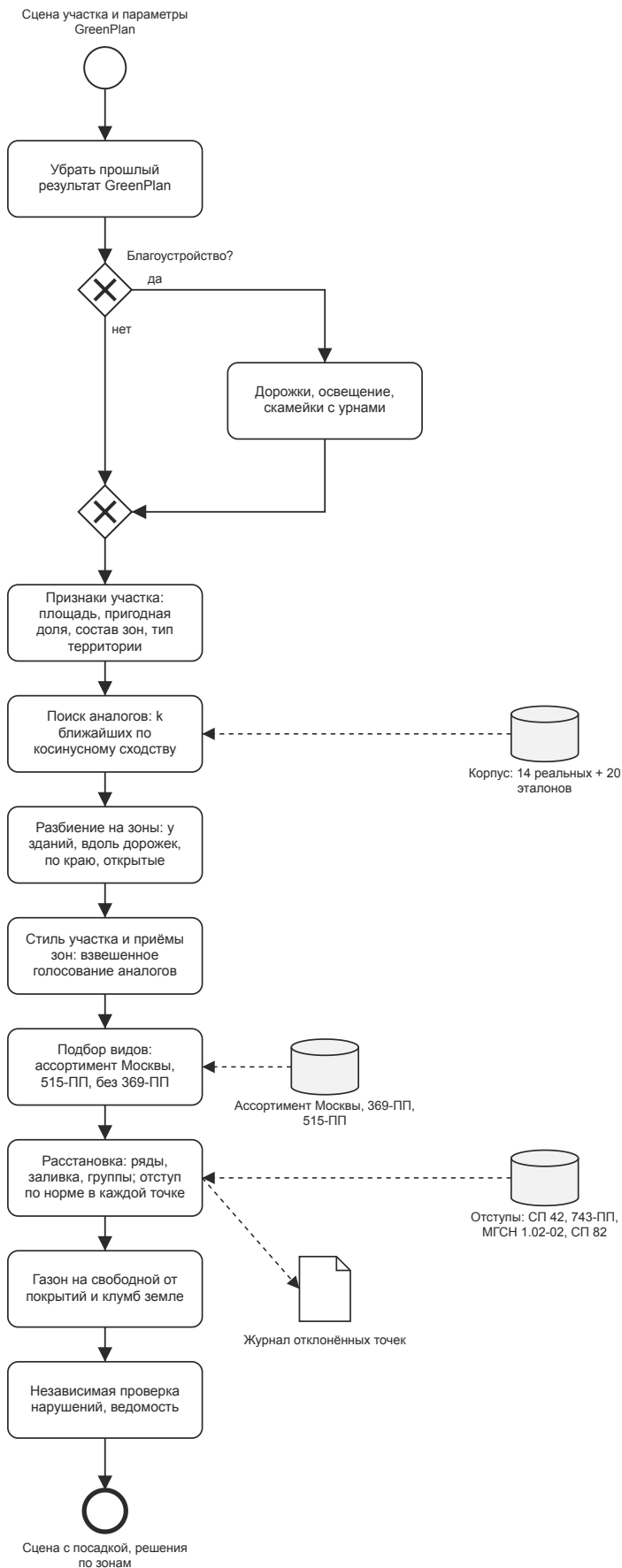


Схема в нотации BPMN, подпроцесс «GreenPlan» из общего процесса. Источник: [bpmn/greencity_process.bpmn](#).

Шаги

№	Шаг	Модуль	Что получается
0	Подготовка	<code>pipeline.py</code>	сцена без прошлого результата GreenPlan
1	Благоустройство (по параметрам)	<code>improvements.py</code>	дорожки, фонари, скамейки с урнами
2	Признаки участка	<code>site_characterization.py</code>	вектор признаков, тип территории
3	Поиск аналогов	<code>pattern_retrieval.py</code> , <code>pattern_corpus.py</code>	k самых похожих проектов корпуса
4	Разбиение на зоны	<code>zone_partitioning.py</code>	геометрические зоны четырёх видов
5	Стиль участка и приёмы зон	<code>pattern_assignment.py</code> , <code>pattern_library.py</code>	стиль, ведущий аналог, приём каждой зоны
6	Подбор видов	<code>species_selection.py</code>	палитра видов участка и основание подбора
7	Расстановка	<code>deterministic_placement.py</code> , <code>core/placement.py</code>	новые деревья и кустарники, журнал отклонений
8	Газон	<code>lawn.py</code>	газон площадью: новый и сохраняемый
9	Проверка и ведомость	<code>violation_report.py</code> , <code>assortment_report.py</code>	нарушения, ведомость по видам
10	Текст и документы	<code>decision_report.py</code> , <code>document.py</code> , <code>explanations.py</code>	текст-обоснование, записка DOCX, файл объяснений

0. Подготовка

Повторный запуск должен заменять прошлый результат, а не добавлять второй слой посадок. Поэтому из сцены убираются объекты и зоны с пометкой `source: "greenplan"` и газон GreenPlan. Существующие объекты и правки пользователя остаются.

1. Благоустройство

Шаг выполняется только если пользователь включил его в параметрах. Благоустройство строится раньше посадок, чтобы посадки соблюдали отступы от него.

- **Дорожки.** Во дворах участка, не больше трёх (`MAX_YARDS`), строится каркас: от подъезда к подъезду и к парковке, если она рядом. Каркас — тот же, что у операции «дизайн двора» ИИ-ассистента (`text_editor/courtyard_layout.py`): минимальное остовное дерево между точками, вынесенными от подъездов вглубь двора. Дорожка становится зоной `pedestrian_path` шириной 1,5 м. От неё отсчитываются отступы посадок (для дерева 0,7 м), её обходит газон, вдоль неё появляются полосы под изгородь. В DXF она попадает на слой `NEW_PATHS`.
- **Освещение.** Фонари ставятся через 20 м зигзагом вдоль новых дорожек и снаружи вдоль существующих. Опоры не ставятся в охранные зоны сетей: фонарь ищет место на другой стороне дорожки или со сдвигом до 8 м. Что не удалось поставить, записывается в `notes`.
- **Скамейки с урнами** ставятся через 30 м вдоль новых дорожек.

2. Признаки участка

`characterize_site(scene)` считает:

- площадь участка;
- пригодную под озеленение площадь: разрешённые зоны (или весь участок) минус запретные и предупреждающие зоны с отступом для дерева;
- долю пригодной площади (`plantable_ratio`);
- число зданий;
- доли площади по типам зон ограничений: здания, дороги, дорожки, газопровод, канализация, водопровод, электросети, детские площадки;
- вытянутость границы: отношение сторон описанного прямоугольника;
- тип территории.

Тип территории определяется правилами — категориями из ассортимента озеленения Москвы, которые различимы по геометрии. Правила проверяются по порядку, срабатывает первое:

Тип	Правило
двор	есть детская площадка и хотя бы одно здание
промышленная, охранная	без зданий, охранные зоны сетей занимают не меньше 40 % площади
улица	есть проезжая часть, стороны границы 3:1 и больше
парк, сквер	без зданий, не меньше 2000 м ² , пригодная доля не меньше 0,7
площадь	без зданий, не меньше 2000 м ² , компактная (стороны меньше 3:1), пригодная доля не больше 0,35
не определён	ни одно правило не сработало

На участках меньше 2000 м² формы и доли слишком шумные, поэтому парк и площадь на них не определяются.

3. Поиск аналогов

Корпус. В корпусе 34 проекта (`locations/`, решения — `data/pattern_corpus.yaml`):

- **14 реальных проектов** из набора кейса. У каждого есть DXF и `design_rationale.md` — выдержка из пояснительной записки о том, какой приём применён и почему.
- **20 синтетических эталонов** (`21_..._primer` — `40_..._primer`). Это типовые городские случаи, которых среди реальных проектов нет или мало: шумозащитная полоса, круглая площадь, двор с детской площадкой, бульвар, озеленённая парковка, сквер над сетями, 10 фрагментов улиц с домами. Их собирает генератор `tools/primers/`, и каждая посадка в них проверена по тем же нормам. Каждый эталон помечен как синтетический.

Признаки проектов корпуса посчитаны заранее: `data/pattern_corpus_features.json` хранится вместе с sha256 исходного DXF. Если DXF изменился, признаки пересчитываются.

Вектор признаков состоит из:

- `plantable_ratio`;
- `log(1 + площадь)` — логарифм, чтобы разброс площадей не заглушал остальное;
- числа зданий;

- долей восьми известных типов зон;
- one-hot типа территории (только если тип определён, а не «не определён»).

Сходство. Каждый признак стандартизуется (z-score) по среднему и стандартному отклонению корпуса. Без этого логарифм площади (порядок 5–12) заглушил бы доли (0–1). Сходство участка с проектом — косинус между стандартизованными векторами. Признак, постоянный по всему корпусу, делится на 1, а не на 0.

Результат — k ближайших проектов (`NeighborMatch(slug, similarity)`), по умолчанию k = 3 (параметр k запроса, от 1 до 9).

Самопроверка: каждый проект корпуса, поданный как новый участок, находит сам себя со сходством 1,0. Это регрессионный тест поиска.

4. Разбиение на зоны

Пригодная площадь делится на геометрические зоны четырёх видов, в порядке приоритета:

Вид зоны	Что это	Для каких приёмов
<code>building_border</code>	полоса 3 м сразу за нормативным отступом от здания	кольцо у здания, изгородь
<code>path_corridor</code>	полоса 3 м вдоль дорожек и проезжей части	ряды, аллеи, изгородь
<code>site_edge</code>	полоса 3 м вдоль границы участка	изгородь по периметру
<code>open_area</code>	всё остальное	площадные приёмы: рощи, сетки, заливка

Место, близкое и к дому, и к дорожке, достаётся дому: у нормы отступа от здания больше вес. Каждый несвязный кусок становится отдельной зоной, чтобы приём назначался ему независимо. Многоугольник с дыркой разрезается на куски без дыр, потому что зона хранит один контур.

5. Стиль участка и приёмы зон

Библиотека приёмов (`pattern_library.py`) — десять параметризуемых приёмов в трёх семействах геометрии:

Приём	Семейство	Стиль	Виды зон
<code>linear_hedge_row</code> — изгородь или ряд вдоль полосы	linear	нейтральный	у здания, вдоль дорожек, по краю
<code>building_ring</code> — кольцо кустарника вокруг здания	linear	нейтральный	у здания
<code>diagonal_rows</code> — диагональные ряды	linear	регулярный	открытые, вдоль дорожек
<code>flowing_rows</code> — волнистые ряды	linear	пейзажный	открытые, вдоль дорожек
<code>formal_bosque_grid</code> — боскет, ортогональная сетка деревьев	linear	регулярный	открытые
<code>concentric_rings</code> — концентрические кольца	linear	регулярный	открытые
<code>triangular_grid_fill</code> — треугольная сетка	area_fill	регулярный	открытые
<code>grove_clusters</code> — рощи	clustered	пейзажный	открытые
<code>poisson_scatter_fill</code> — свободный разброс	area_fill	пейзажный	открытые
<code>generic_fill</code> — заливка без замысла	area_fill	нейтральный	любые

Решение принимается на двух уровнях, чтобы у участка был единый стиль. Если выбирать приём для каждой зоны отдельно, в одном дворе оказываются волны рядом с диагоналями.

Уровень 1 — стиль участка. Голосуют к самым похожим проектам (сходство не ниже `MIN_VOTE_SIMILARITY = 0.2`), у которых для видов зон этого участка есть решение регулярного или пейзажного стиля. Проект только с нейтральными решениями место избирателя не занимает. Голос весит столько, сколько сходство. Побеждает стиль с наибольшей суммой весов. Ведущий аналог — самый похожий проект победившего стиля. Если пользователь задал стиль в параметрах, голосования нет. Тогда ведущий аналог — самый похожий проект этого стиля.

Уровень 2 — приёмы зон. Для каждого вида зоны голосуют к самым похожим проектам с решением для этого вида в стиле участка или нейтральным. Голосуют только проекты со сходством не ниже порога. Заливка без замысла голосует, только если других решений нет. Зоны одного вида делятся между приёмами по площади, пропорционально весу голосов: крупнейшая зона достаётся самому весоному приёму, каждая следующая — приёму, который сильнее всех недобрал свою долю.

Происхождение решения записывается в `ZoneAssignment`:

- `source_project` — самый похожий проект среди голосовавших за приём зоны;
- `confidence` — доля избирателей, голосовавших за этот приём;
- `site_style`, `lead_project` — общее решение участка.

Нет аналогов. Если избирателей нет, зона получает типовой приём стиля: для открытых зон регулярного участка — треугольную сетку, пейзажного — рощи, для полос — изгородь или кольцо у здания. Такое решение идёт с `confidence = 0` и `source_project = None`. В записке и на панели оно помечено «типовое решение — проверить».

Порог 0,2 подобран на корпусе: доля площади зон на типовом приёме — 5,6 %, и ни один тестовый участок не остаётся без аналогов совсем.

6. Подбор видов

Для каждой пары (вид зоны, приём) подбираются виды деревьев и кустарников — один набор на весь участок. Например, изгородь вдоль всех дорожек делается из одного вида.

Фильтры по нормативным документам:

1. Вид есть в ассортименте для озеленения Москвы (`data/norms/assortment-msk`) и отмечен «+» для типа территории участка. Тип «не определён» считается дворовой территорией.
2. Вида нет в перечне инвазивных растений ППМ 369-ПП. Каталог их и так не содержит, здесь проверка повторная.
3. Коды ограничений ассортимента:
 - код 2 (не высаживать вблизи детских площадок) — такой вид не сажается на участке с детской площадкой;
 - код 5 (чувствителен к реагентам и выхлопам) — не в полосы вдоль дорожек и проездов;
 - код 7 — самосев; такие виды не сажаются, потому что границ особо охраняемых территорий в DXF нет.
4. Колючие виды — не вдоль дорожек (СП 82.13330.2016, п. 9.22).
5. Породы с широкой кроной — не в полосу у здания: им нужно 10 м от стены (ППМ 743-ПП, прим. 3 к табл. 3.6.1).

Роль и форма. У каждого приёма есть роль для деревьев и для кустарников: группы морфологических форм и число видов. Аллея, изгородь и боскет делаются из одного вида, роща и свободная посадка — из смеси разных родов. Форма берётся по морфологическому архетипу вида (`data/plant_archetypes/species_catalog.json`).

Порядок выбора:

1. Базовый ассортимент ППМ 515-ПП (для дворов).
2. Классы ассортимента: основной, затем дополнительный, затем перспективный.
3. Стабильный хеш участка: разные участки получают разные виды, один и тот же участок — всегда одни и те же.

Единая палитра участка. Пары (вид зоны, приём) обрабатываются по убыванию площади. Крупнейшие задают палитру, следующие берут из неё виды подходящей формы и добавляют новые, только пока в палитре меньше четырёх видов деревьев и четырёх видов кустарников. Предел мягкий: роль, которой по форме не подходит ни один вид палитры, всё равно получает свой вид.

Предпочтительные виды пользователя ставятся первыми в любой роли своей категории, без ограничения по форме. Нормы для них действуют как для всех. Почему вид не использован, пишется в `notes`.

Если ни один вид каталога не прошёл фильтры (например, в тестах с синтетическим каталогом), каталог берётся как есть, и основание помечено «вне ассортимента».

7. Расстановка

Приём и зона превращаются в точки посадки одним алгоритмом на семейство, а не отдельным кодом на каждый приём:

- **linear** — точки с шагом вдоль линии:
 - формы линии: прямая по длинной оси зоны, синусоида, диагональ, контур зоны (кольцо у здания), вложенные кольца;
 - на широкой открытой зоне — столько параллельных рядов с шагом 6 м, сколько помещается, но не больше 10;
 - шаг кустов 1,4 м, деревья в 8,5 раза реже, если приём не задал свой шаг (у боскета шаг квадратной сетки);
 - деревья сажаются раньше кустов, иначе кусты заняли бы всю линию.
- **area_fill** — кандидаты по решётке на всей площади зоны и равномерный отбор (`pick_spread`):
 - шаг кустов 6 м, деревьев — вдвое больше;
 - треугольная сетка берёт точки прямо с решётки: ей нужен видимый регулярный узор.
- **clustered** — центры групп через 8 м, одна группа на 200 м², у каждого центра — плотная группа одного вида с шагом 2 м.

Проверка нормы в каждой точке. Точка проверяется во время расстановки, а не фильтром после.

Планировщик `Placer` (`core/placement.py`) проверяет по каждой точке:

- внутри участка, не ближе 0,5 м к его границе;
- вне запретных и предупреждающих зон, расширенных на нормативный отступ с учётом породы (`core/setback_norms.py`, таблица ниже);
- не ближе 1 м к другим объектам; для пар с нормой — не ближе нормы (дерево — 4 м от опоры освещения, СП 42 табл. 9.1).

Кандидаты заранее очищаются от мест, занятых существующими объектами. Иначе на участке с сотнями старых деревьев центр рощи почти всегда попадал к старому дереву, и группа не вставала.

Журнал отклонений. Каждая точка, отклонённая по норме (сеть, здание, порода у здания или теплосети, опора освещения, колючие у дорожек), записывается в журнал (`RejectionLog`) с правилом и пунктом НПА. Подробно записываются первые 300 на участок, остальные только считаются. Отказы по зазору между соседними объектами — правило планировщика, а не норма, поэтому они только считаются.

Пределы объёма:

- не больше 400 деревьев и 400 кустов на зону; у рядов предел достигается увеличением шага, чтобы ряды оставались по всей зоне;
- не больше 60 групп на зону;
- не больше 10 000 новых посадок на участок; сверх этого — равномерное прореживание с сохранением доли каждой зоны.

Вид в 3D. Деревья получают размер с разбросом (15 % в свободных посадках, 6 % в регулярных) и случайный поворот. Разброс детерминированный: сид — `id` и место объекта. Высота модели подгоняется под реальную высоту вида из каталога.

8. Газон

В проектах газон считают площадью: в ведомости по форме 9 ГОСТ 21.508 это м^2 . Поэтому газон строится многоугольниками после посадок:

- **где:** разрешённые зоны (слои газона и вычисленная парсером открытая земля; если их нет — весь участок) минус твёрдые покрытия и сооружения, минус клумбы кустарника (круг кроны, соседние кусты ряда сливаются);
- **над сетями — можно:** таблица 9.1 СП 42.13330 и таблица 3.6.1 ППМ 743-ПП нормируют расстояния только для деревьев и кустарников;
- **очистка:** полосы уже 0,5 м и куски меньше 5 м^2 отбрасываются, контур упрощается с точностью 0,25 м;
- **статус:** газон на существующем газоне исходного плана — `existing` (сохраняется), остальной — `new` (устройство). В ведомость и DXF (слой `NEW_LAWN`) идёт только новый;
- **вид:** «Газон обыкновенный», посев из устойчивой травосмеси (ППМ 515-ПП, табл. 4).

9. Проверка нарушений и ведомость

`find_violations` заново проверяет **все** объекты сцены, и существующие, и новые, против всех зон ограничений. Код расстановки для этого не используется. Для новых посадок это независимое подтверждение, что нарушений нет, а нарушения существующих объектов не скрываются. Кандидаты отбираются через STRtree по зонам, расширенным на худший отступ, а точное расстояние считается по исходной геометрии.

Ведомость (`assortment_report.py`) — количество новых деревьев и кустарников по видам (шт.), газон (м^2) и отдельно благоустройство.

10. Текст и документы

- **Текст-обоснование** (`POST /api/greenplan/report, decision_report.py`). Решения зон сводятся по (вид зоны, приём, аналог), и YandexGPT 5.1 Pro пересказывает сводку связным текстом. Модели запрещено добавлять виды, площади и проекты, которых нет в сводке. Без ключа или при недоступной модели ответ 200 с `report_error`; расстановка от этого не зависит.

- **Пояснительная записка** (`POST /api/greenplan/document` , DOCX). Её разделы:
 - сведения об участке;
 - нормативная база и применённые отступы;
 - решения с происхождением («по аналогии с проектом Х» или «типовое решение — проверить»);
 - ведомость по форме 9 ГОСТ 21.508-2020;
 - проверка норм;
 - технические требования по СП 82.13330.2016;
 - ограничения и допущения.

Текст ИИ идёт только приложением с пометкой «проверить». - **Файл объяснений** (`POST /api/greenplan/explanations` ,JSON или CSV). Описан в [документации решения](#).

Нормативные отступы

`core/setback_norms.py` , СП 42.13330.2016 п. 9.6 табл. 9.1 и совпадающая с ней табл. 3.6.1 ППМ 743-ПП. Расстояние считается от оси растения до края зоны.

Ограничение	Дерево	Кустарник
Здание	5,0 м	1,5 м
Проезжая часть	2,0 м	1,0 м
Пешеходная дорожка	0,7 м	0,5 м
Электрокабель, кабель связи	2,0 м	0,7 м
Теплосеть	2,0 м	1,0 м
Газопровод, канализация	1,5 м	не нормируется
Водопровод	2,0 м	не нормируется
Опора освещения	4,0 м	не нормируется

Правила по породе только ужесточают таблицу:

- липа, клён, дуб, каштан, тополь — 10 м от здания (ППМ 743-ПП, прим. 3 к табл. 3.6.1);
- колючие кустарники — 2 м от дорожек и детских площадок (СП 82.13330.2016, п. 9.22);
- породы, чувствительные к прогреву, — 2–4 м от теплосети (МГСН 1.02-02, п. 4.2.8);
- экспертные оценки — отдельно, с пометкой «экспертная оценка, не норма акта».

У зон без табличного значения (трансформатор, парковка, площадка, ЛЭП) используется ширина охранной зоны из разметки.

Свойства алгоритма

- **Детерминированность.** Одна и та же сцена с теми же параметрами даёт тот же результат: случайность только от сидов, привязанных к участку и месту.
- **Прослеживаемость.** У каждой посадки известны:
 - зона и приём;
 - проект-аналог и стиль;
 - основание подбора вида;

- ближайшие ограничения с нормой и пунктом НПА;
- id, общий для сцены, DXF (XDATA) и файла объяснений.
- **Без обучения.** При корпусе в десятки проектов обучение не на чем проверить, поэтому решение переиспользует то, что архитекторы уже сделали на похожих участках. Добавить проект в корпус — значит положить DXF с описанием решений и запустить `make corpus-features`.
- **Честность.** Нарушения не скрываются. Типовые решения без аналогов помечены. Невыполненные параметры попадают в `notes`. Отклонённые по норме точки попадают в журнал.

Ограничения

- Корпус маленький. Реальные проекты описаны частично, в основном одним-двумя видами зон, поэтому часть решений берётся у синтетических эталонов.
- Реальные участки сами входят в корпус. Для участка из набора кейса ближайшим аналогом может оказаться он сам.
- Решения по существующим деревьям (сохранить, пересадить, вырубить) и компенсационный расчёт не делаются: в данных кейса нет перечётных ведомостей для этого.
- Ширина полос у зданий, дорожек и края (3 м) и шаги посадки подобраны по реальным проектам, а не взяты из норм.

Подробнее — в разделе «Ограничения, допущения, известные ошибки» документации решения.

GreenCity

Сервис для автоматического проектирования озеленения города с учётом расположения подземных коммуникаций и городской среды (хакатон «Лидеры цифровой трансформации 2026», команда «Вороны»).

Документация

- [docs/SOLUTION.md](#) — документация решения: назначение и границы MVP, схема процесса BPMN, интерпретируемость, методы и библиотеки, проверка выходного DXF в nanoCAD и на Linux, ограничения и известные ошибки, примеры — начните отсюда. PDF-версия — [docs/pdf/](#).
- [docs/GREENPLAN_ALGORITHM.md](#) — алгоритм GreenPlan по шагам.
- [docs/TECHNICAL_OVERVIEW.md](#) — технический обзор: архитектура, модули, API, ИИ-ассистент, экспорт, тесты, частые вопросы.
- [examples/](#) — выходные DXF, файлы объяснений и пояснительные записки для трёх участков из набора кейса.
- [backend/README.md](#) — эндпоинты бэкенда, алгоритм автогенерации озеленения, параметры.
- [data/norms/README.md](#) — нормативные акты в машиночитаемом виде, что извлечено/проверено.
- [observability/README.md](#) — метрики, логи, дашборды Grafana.
- [frontend/public/models/README.md](#) — 3D-модели, как подключить свой пак.
- [docs/CASE_BRIEF.md](#) — исходное описание кейса от организаторов хакатона ЛЦТ.
- [docs/SOURCECRAFT_DEPLOY.md](#) — деплой backend в Yandex Cloud через SourceCraft CI/CD (`.sourcecraft/ci.yaml`), какие шаги нужно сделать руками перед первым запуском.

Структура репозитория

<code>locations/</code>	14 реальных документированных проектов + 20 синтетических эталонов (21–40): <code><slug>/<slug>.dxf + design_rationale.md</code> — корпус GreenPlan для поиска похожих проектов; <code>location_old/</code> — 6 синтетических участков для юнит-тестов; <code>00_empty_plot/</code> — пустой участок для ручной расстановки
<code>data/</code>	<code>norms/</code> — выписки из нормативных актов (<code>data/norms/README.md</code>); <code>pattern_corpus.yaml</code> — какой приём в какой зоне применён в каждом проекте корпуса; <code>plant_archetypes/</code> — справочник видов и 3D-архетипов
<code>parser/</code>	DXF → сцена JSON: <code>parse_dxf.py</code> (сборка и командная строка) + <code>dxf_parsing/</code> (правила слоёв, зоны и граница, здания и точечные объекты)
<code>converters/</code>	GeoJSON / SHP / DWG → DXF (для внешних источников данных)
<code>backend/</code>	FastAPI (<code>main.py</code>) — подробности <code>backend/README.md</code> :
<code> api/</code>	эндпоинты: <code>editor.py</code> , <code>greenplan.py</code> , <code>accounts.py</code>
<code> core/</code>	сцена (<code>schemas</code>), нормы отступов, каталог, инвазивные виды, <code>Placer</code>
<code> greenplan/</code>	GreenPlan: признаки участка, зоны, поиск аналогов, паттерны, подбор видов, расстановка, нарушения, ведомость, отчёт, записка DOCX
<code> text_editor/</code>	правка плана текстом через LLM и дизайн двора (<code>design_area</code>)
<code> generation/</code>	генератор по сетке (<code>/api/generate-greenery</code>)
<code> exchange/</code>	экспорт DXF, конвертация папки DWG, мост к <code>parser/</code>
<code> accounts/</code>	регистрация, вход, проекты; <code>storage/</code> — MongoDB и Redis
<code> monitoring/</code>	JSON-логи и метрики Prometheus
<code> tests/</code>	тесты по тем же пакетам + e2e/ (сквозные сценарии через HTTP)
<code>frontend/</code>	React + TypeScript + react-three-fiber: 3D-сцена и редактор (<code>src/pages/editor/</code> — страница редактора и её панели)
<code>docs/</code>	документация, схема процесса BPMN (<code>docs/bpmn/</code>), PDF (<code>docs/pdf/</code>)
<code>examples/</code>	примеры результата для трёх участков из набора кейса
<code>tools/</code>	эталон корпуса, примеры, проверка выходного DXF, сборка схем и PDF

Быстрый старт

Обе LLM проекта – в Yandex AI Studio, с одним ключом и каталогом (`YANDEX_CLOUD_API_KEY` , `YANDEX_CLOUD_FOLDER`): ИИ-ассистент – Qwen3 235B (`YANDEX_CLOUD_MODEL`), текст-обоснование GreenPlan – YandexGPT 5.1 Pro (`YANDEX_CLOUD_REPORT_MODEL`). Для ассистента можно переключиться на Gemini (`LLM_PROVIDER=gemini`). Скопируйте шаблон и заполните значения – сам `.env` в git не попадает:

```
cp .env.example .env
```

Без `.env` всё остальное работает, а правка текстом отвечает понятной ошибкой; без ключа Yandex GreenPlan расставляет посадки и формирует пояснительную записку, но без текста-обоснования. То же самое с аккаунтами: без MongoDB/Redis (см. ниже) регистрация и вход отвечают понятной ошибкой, а не роняют бэкенд – сам редактор (DXF, генерация, правка текстом) от них не зависит вовсе.

Вариант А: Docker (основной способ запуска)

```
docker compose up -d --build
```

Поднимет весь проект разом, восемь сервисов: frontend (`http://localhost:5173` – редактор, API и Swagger), backend, MongoDB (аккаунты и проекты, `backend/storage/db.py`), Redis (ограничение частоты входа, кэш каталога, сессии по refresh-токенам, `backend/storage/cache.py`) и стек наблюдаемости – Prometheus, Loki, Promtail и Grafana (`http://localhost:3001`, логин/пароль `admin/admin`) с готовыми дашбордами метрик и логов, подробности – `observability/README.md`. У каждого сервиса – `healthcheck` и `depends_on` по нему же (например, backend стартует, только когда mongo/redis реально приняли первое подключение, а не просто "контейнер запущен"; grafana – когда prometheus и loki реально готовы), и `restart: unless-stopped` – сервис, упавший сам по себе (а не остановленный командой), Docker поднимает заново автоматически. Код смонтирован volume'ами – правки в `backend/`, `parser/`, `frontend/` подхватываются на лету (`uvicorn --reload`, `vite dev server`), без пересборки образа. Это касается только кода: **новые зависимости** в `requirements.txt` так не подхватываются – после их изменения нужен `docker compose up -d --build backend`, иначе бэкенд упадёт на импорте.

Наружу открыты только два адреса:

Адрес	Что там
<code>http://localhost:5173</code>	редактор; <code>/api/...</code> и Swagger (<code>/docs</code>) dev-сервер фронтенда проксирует на бэкенд
<code>http://localhost:3001</code>	Grafana

Бэкенд, MongoDB, Redis, Prometheus и Loki доступны только внутри сети контейнеров, напрямую с хоста к ним не обратиться. Браузер ходит к API с того же адреса, что и к сайту, поэтому CORS на бэкенде выключен (подробнее – раздел «Сеть и защита API» ниже).

Это дев-режим: по умолчанию backend работает одним процессом с `--reload` (`UVICORN_WORKERS=1`, `UVICORN_RELOAD=1` в `docker-compose.yml`), совмещать `--reload` с несколькими процессами uvicorn не умеет. Для нагрузочных прогонов и продакшена стоит поднять до числа свободных ядер (`UVICORN_WORKERS=4`): генерация озеленения и разбор DXF – это чистая CPU-геометрия на shapely, потоками она не распараллеливается (весь Python-код в процессе идёт через один GIL), и настоящий параллелизм здесь дают только отдельные процессы. Внутри процесса обработчики асинхронные: тяжёлые расчёты идут по одному в пуле потоков, а лёгкие запросы (каталог, вход, health) и ожидание ответа LLM их не ждут.

Прямые ссылки на готовые дашборды Grafana (логин/пароль `admin/admin` при первом переходе):

- Обзор (HTTP + бизнес-метрики): <http://localhost:3001/d/greencity-overview>
- Логи: <http://localhost:3001/d/greencity-logs>

Если при первой сборке `docker pull` какого-то образа падает с `authentication required/toomanyrequests` — это временное ограничение частоты анонимных запросов к Docker Hub (не проблема самого проекта, проверено — повторный `docker pull` того же образа сразу проходит); помогает повторить команду через пару минут или выполнить `docker login` (у авторизованных лимит выше).

Вариант A2: Docker на сервере (продакшен-режим)

```
make docker-prod-up # docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
```

Поверх того же `docker-compose.yml` (`docker-compose.prod.yml`):

- **frontend** — статическая сборка за nginx (`frontend/Dockerfile.prod`) на порту 80 вместо vite dev server; `/api/` nginx проксирует на backend, сайт и API на одном адресе;
- **backend** — код из образа, без смонтированных каталогов и без `--reload`, `UVICORN_WORKERS` процессов (3); наружу не открыт, только через nginx; Swagger выключен (`API_DOCS=0`);
- **mongo** — кеш ограничен 0,5 ГБ;
- образ backend — без исходных материалов участков (`locations/*/data`, 56 МБ PDF и сканов): бэкенду нужны только DXF и `design_rationale.md`.

Ответы API сжимаются (gzip): сцена реального участка — 1–13 МБ JSON, в сжатом виде в 5–6 раз меньше.

Продакшен-режим рассчитан на сервер **4 vCPU (2 ГГц) / 8 ГБ ОЗУ / 40 ГБ SSD** (LLM — облачные, по API):

Сервис	Память	CPU
backend, 3 процесса	до 4 ГБ (до ~750 МиБ на процесс на крупном участке; разбор DWG — до 2 пачек одновременно, ~0,75 ГБ + 3 процесса по ~0,5 ГБ на пачку)	3 ядра
MongoDB, кеш 0,5 ГБ	до 1 ГБ	—
Redis	до 256 МБ	—
frontend (nginx)	до 128 МБ	—
мониторинг и система	~1,4 ГБ	четвёртое ядро

Узкое место — процессор: один тяжёлый запрос занимает одно ядро, при 2 ГГц GreenPlan считается 2–10 с на обычном участке и до ~30 с на участке в 2 км². Три процесса — три тяжёлых запроса одновременно; четвёртое ядро оставлено базе, nginx и мониторингу, чтобы сайт отвечал под нагрузкой. Логи контейнеров ограничены (3 файла по 10 МБ на сервис), старые образы и кеш сборки после обновлений удаляет `make docker-prune`.

На другом сервере число процессов меняется переменной (около 750 МиБ памяти на процесс):

```
UVICORN_WORKERS=2 make docker-prod-up
```

Вариант B: локально

1. Python-окружение (parser, converters, backend)

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Для конвертации `.dwg` дополнительно нужен LibreDWG (`brew install libredwg` / `apt install libredwg-tools`).

2. Backend

```
source .venv/bin/activate
cd backend
uvicorn main:app --reload --port 8000
```

Swagger — `http://localhost:8000/docs` (или через фронтенд — `http://localhost:5173/docs`).

Эндпоинты: `POST /api/parse` — принимает `.dxf`, отдаёт `{boundary, restrictions, objects, windows, canopies, meta}`; `POST /api/parse-dwg` — то же самое, но на входе папка из нескольких `.dwg`: каждый файл конвертируется в DXF через `dwg2dxf` / LibreDWG и сливается в один документ на бэкенде, файлы, которые не удалось прочитать, попадают в `dwgConversionWarnings` ответа, а не роняют весь запрос; `POST /api/generate-greenery` — принимает сцену того же формата (+ опциональные `species/grid_spacing_m/min_tree_spacing_m/bush_grid_spacing_m/min_bush_spacing_m/lawn_patch_size_m/include_trees/include_bushes/include_lawn` через query string), возвращает её же с добавленными деревьями, группами кустов и газоном (детерминированный генератор по сетке, без ML). Полное описание алгоритма и параметров — `backend/README.md`. `GET /api/catalog` — каталог видов посадок и МАФ. `POST /api/edit-with-text` — принимает `{scene, instruction, history}`, правит план текстом через LLM. `POST /api/export-dxf` — принимает сцену, отдаёт `.dxf`: исходный чертёж без изменений плюс слои результата `NEW_*` (GreenPlan) и `USER_*` (правки пользователя) в его координатах (`backend/exchange/dxf_overlay.py`). `POST /api/greenplan/explanations` — объяснение каждой посадки со ссылкой на НПА и пункт (JSON или CSV).

3. Frontend

```
cd frontend
npm install
npm run dev
```

Открыть `http://localhost:5173`, загрузить DXF (например, из `locations/`), кликнуть по дереву/кусту/лавке/фонарю — выбрать и перетащить. Dev-сервер проксирует `/api/...` на бэкенд `http://localhost:8000`; другой адрес задаётся переменной `API_PROXY_TARGET`.

Сеть и защита API

- **Доступ только через фронтенд.** Бэкенд слушает только сеть контейнеров, наружу его отдаёт прокси фронтенда: vite в дев-режиме, nginx в продакшене. CORS выключен; если фронтенд живёт на другом домене, разрешённые адреса задаются в `CORS_ALLOW_ORIGINS`. IP клиента бэкенд берёт из `X-Forwarded-For`, который выставляет прокси.
- **Swagger** открыт в дев-режиме (`API_DOCS=1`), в продакшене выключен.
- **Тяжёлые эндпоинты** защищены тремя механизмами. Это разбор DXF и DWG, GreenPlan, экспорт, правка текстом и генерация по сетке.

- **Лимит частоты с одного IP** за 10 минут, по группам: загрузка DXF — 30, пачка DWG — 10, GreenPlan (генерация, записка, объяснения) — 60, запросы к LLM (ассистент, текст-обоснование) — 30, экспорт — 30, генерация по сетке — 30. Сверх лимита ответ 429 с заголовком `Retry-After`. Счётчики лежат в Redis; без Redis лимиты не действуют.
- **Размер запроса:** загрузка чертежей — до 200 МБ, прочие запросы — до 64 МБ, не больше 60 файлов `.dwg` за раз. Сверх лимита — 413.
- **Очередь расчётов:** один тяжёлый расчёт на процесс `uvicorn`, до 8 ждущих. Если очередь полна или ожидание дольше 2 минут — ответ 503 с `Retry-After`. Лёгкие запросы в очереди не стоят.
- **LLM:** запросы к моделям асинхронные, с таймаутом 60–120 с и одним повтором. Пока модель отвечает, сервер обслуживает других.

Все пороги настраиваются переменными окружения (`.env.example`): `RATE_LIMIT_MULTIPLIER`, `MAX_UPLOAD_MB`, `MAX_BODY_MB`, `MAX_DWG_FILES`, `HEAVY_CONCURRENCY`, `HEAVY_QUEUE_MAX`, `HEAVY_WAIT_S`.
 Подробнее — `docs/TECHNICAL_OVERVIEW.md`, раздел 2.1.

Тесты

```
make test      # все тесты backend/ и parser/, параллельно (pytest-xdist)
make test-e2e  # только сквозные сценарии (backend/tests/e2e/)
make test-cov  # всё + отчёт о покрытии
```

Около 1080 тестов, прогон около минуты. Тесты лежат по тем же пакетам, что и код (`backend/tests/core/`, `greenplan/`, `text_editor/`, ...; `parser/tests/`). Сквозные тесты (`backend/tests/e2e/`) гоняют настоящее приложение через HTTP на реальных DXF из `locations/`: загрузка → GreenPlan → текст-обоснование → пояснительная записка → экспорт DXF → повторная загрузка; загрузка → правка текстом → генерация → экспорт; регистрация → проект → сохранение результата → выход. Подменены только внешние границы: MongoDB и Redis (in-memory), облачные LLM Yandex AI Studio — правка текстом и текст-обоснование (фиксированные ответы). Ни один тест не обращается к платному API. Качество ИИ-ассистента на настоящей модели проверяет отдельный инструмент `tools/ai_editor_eval/run.py` (платные запросы, запускается вручную; см. `docs/TECHNICAL_OVERVIEW.md`, раздел 5.2).

CI (GitHub Actions и SourceCraft) запускает `ruff`, `oxlint`, проверку типов TypeScript и все тесты.

Линтеры

```
make lint      # всё сразу: Python + фронтенд
make lint-py   # только Python — ruff, конфиг в pyproject.toml
make lint-web  # только фронтенд — oxlint, конфиг в frontend/.oxlintrc.json
make lint-fix  # автоисправление того, что чинится автоматически
```

Обе цели возвращают ненулевой код при любой находке (у `oxlint` для этого включён `--deny-warnings`), поэтому `make lint` годится как гейт в CI. Линтер для Python ставится в `venv` из `requirements-dev.txt` — отдельно от `requirements.txt`, чтобы не тащить его в backend-образ.

Форматтер (`ruff format`) намеренно не подключён: в конфигах парсера и генераторов (`POLYGON_RULES` и т.п.) таблицы выровнены вручную для читаемости, автоформат бы их развалил.

Что умеет прототип сейчас

- Парсинг DXF в структурированную сцену: граница участка, здания (с фасадами — окна/козырьки/подъезды), зоны ограничений (коммуникации, дороги, парковки и т.д.), точечные объекты озеленения
- 3D-просмотр сцены с подсветкой зон по степени ограничения (запрещено / предупреждение / разрешено)
- Пак реальных 3D-моделей (`frontend/public/models/`, подробности и как добавить свой — README там же): 31 дерево и 15 МАФ (шезлонги, лавки, стол, стулья, табурет); вид из каталога без модели рисуется примитивом-заглушкой по габаритам — штатный режим
- Отступы посадок по СП 42.13330.2016 (табл. 9.1) и ППМ 743-ПП (табл. 3.6.1): разные для дерева и кустарника (от здания — 5 м и 1.5 м, от трубопроводов и кабелей — 1.5–2 м), плюс правила по породе — липе и клёну 10 м от здания, колючим кустарникам 2 м от дорожек (СП 82), чувствительным к прогреву породам до 4 м от теплосети (МГСН 1.02-02). Кольца отступов видны вокруг зданий и проверяются при перетаскивании
- Редактирование расположения деревьев/кустов/лавок/фонарей с проверкой нарушений отступов в реальном времени
- Экспорт отредактированного расположения объектов в JSON
- Экспорт итогового плана обратно в DXF (кнопка "Экспорт DXF" → `/api/export-dxf`): результат дописывается поверх исходного чертежа — исходные слои, подписи, блоки и координаты не меняются, посадка GreenPlan — на слоях `NEW_*`, правки пользователя — `USER_*`, у каждой сущности результата XDATA с id посадки
- Объяснение каждой посадки со ссылкой на пункт НПА (кнопки «Объяснения посадок: JSON / CSV» на панели GreenPlan, `backend/greenplan/explanations.py`): вид, приём и аналог, основание подбора вида, ближайшие ограничения с расстоянием и нормой, отклонённые по нормам точки, зоны запрета посадки
- Конвертация GeoJSON/SHP/DWG в DXF для импорта внешних геоданных (Мосгеотрест, `data.mos.ru`)
- Кнопка GreenPlan — автоозеленение по аналогии с похожими реализованными проектами (раздел ниже): приём озеленения для каждой зоны участка, виды из ассортимента Москвы для типа территории, расстановка с проверкой норм, газон в м² (на сцене, в ведомости и в DXF на слое `NEW_LAWN`), текст-обоснование и пояснительная записка в DOCX с ведомостью по форме 9 ГОСТ 21.508
- Параметры GreenPlan перед запуском: стиль участка (авто, регулярный, пейзажный), что сажать (деревья, кустарники, газон), предпочтительные виды деревьев и кустарников (в пределах норм и ассортимента) и благоустройство — дорожки во дворах, освещение, скамейки с урнами; повторный запуск заменяет прошлый результат
- Загрузка папки проекта с `.dwg`-файлами (конвертация на сервере через LibreDWG, `backend/exchange/dwg_batch_converter.py`)
- Генератор по сетке `/api/generate-greenery` — деревья, группы кустов и газон плитками внутри зон газона с нормативными отступами (`backend/generation/`). В интерфейсе кнопки для него нет (его заменил GreenPlan), доступен через API и Swagger (`/docs`)
- Виды из перечня инвазивных растений ППМ 369-ПП исключены из каталога: их не предложат ни GreenPlan, ни правка текстом, ни панель «Добавить объект»
- Правка плана текстом через LLM (Т3: корректировки в текстовом формате). Задача разбита на два уровня: модель выбирает намерение, виды и параметры, а координаты считает детерминированный планировщик (`backend/core/placement.py`). Планировщик сам берёт шаг по размеру вида и держит отступы по нормам: от зданий, сетей, дорожек, парковок, фонарей (дерево — 4 м от опоры освещения) и подъездов; объект с нарушением сдвигает в ближайшее допустимое место (не дальше 10 м), а если такого нет — отклоняет с причиной (`backend/text_editor/service.py`). Операции:

- Точечные: `add/remove/move/rotate` — для конкретных объектов.
- Ряды и группы: `place_along` (вдоль дорожек/фасадов/парковок/площадок/периметра), `place_in_area` (у точки или равномерно по области), `cover_area` (сплошной ковёр газона/цветника — в отличие от `place_in_area`, плитки укладываются встык, а не редкой россыпью), `line_of` (линия объектов между двумя произвольными точками/объектами/целями — обобщение `connect`), `enclose` (кольцо вокруг объекта/цели/точки).
- Правка существующих объектов: `remove_where` (можно без списка типов — «очисти эту зону», чем бы она ни была занята), `replace_where` (замена вида без смены места), `thin_out` (прореживание слишком плотной посадки), `resize` (масштаб), `face` (разворот к точке/объекту), `align_along` (подравнивать сбившийся ряд), `duplicate_near` (копия рядом с другим местом), `set_count` (довести число объектов до N).
- `connect` — дорожка между двумя точками/объектами/целями (например «от подъезда к фонтану», «сеть дорожек — с парковкой»). Прямую, перекрытую зданием, планировщик не отклоняет — сначала пробует обойти препятствие по кратчайшему пути (граф видимости вокруг углов препятствий + Дейкстра, `placement_geometry.shortest_path`).
- `define_zone` — выделить именованную зону («детская зона», «здесь ничего не сажать»): круг вокруг точки/объекта/цели, сразу попадает в зоны ограничений (видна на плане, учитывается всеми правками) и адресуется по имени в `target` любой операции и в `area` у `place_in_area/cover_area` — так решается «выдели зону под сад и посади там кусты».
- `design_area` — полный дизайн двора одной операцией: каркас дорожек (от подъезда к подъезду, с выходом на парковку, если она рядом), фонари и скамейки с урнами вдоль дорожек, живая изгородь по краю, деревья вразброс по свободной площади (виды по умолчанию — из ассортимента Москвы для дворов, как у GreenPlan); фонтан и площадь — только по явной просьбе и там, где для них реально есть место (`backend/text_editor/courtyard_design.py`).
- `run_greenplan` — «озелени участок в регулярном стиле, побольше лип»: модель выбирает параметры GreenPlan, а фронтенд запускает его с панелью решений и пояснительной запиской.

Виды можно называть по-русски («липы», «сирень», «клён остролистный») — названные в просьбе виды всегда попадают в контекст модели. Ассистент помнит последние правки чата («убери их», «то же самое у второго дома»), а последнюю правку можно отменить кнопкой в чате.

Провайдер выбирается переменной `LLM_PROVIDER` в `.env` — `yandex` (по умолчанию, Yandex AI Studio, модель Qwen3 235B) или `gemini`, оба через OpenAI-совместимый API (`backend/text_editor/service.py`, `backend/core/yandex_ai.py`); ответ приходит за несколько секунд - Запуск в Docker (`docker compose up -d --build`) — healthcheck на каждом сервисе, зависимости по факту готовности (`depends_on: condition: service_healthy`), автоматический перезапуск упавшего контейнера (`restart: unless-stopped`) - Наблюдаемость: метрики Prometheus (стандартные HTTP + бизнес-метрики — сколько распарсено/сгенерировано/экспортировано, исходы правки текстом и входа/регистрации) и структурированные JSON-логи с `request_id`, дашборды Grafana готовы из коробки — поднимаются вместе со всем остальным одной командой `docker compose up`, подробности `observability/README.md`

GreenPlan — озеленение по прошлым проектам

Детерминированный конвейер, который предлагает озеленение участка не с нуля, а по аналогии с тем, что реально делали архитекторы на похожих проектах — без обучения модели и без координат от LLM. Алгоритм подробно — [docs/GREENPLAN_ALGORITHM.md](#). Статус по этапам:

Этап	Что делает	Статус
0. Справочники	Нормативы отступов, ассортимент, инвазивные виды в машиночитаемом виде	Готово (<code>data/norms/</code> , 10 актов)
1. Импорт DWG→DXF	Конвертация + классификация слоёв по подстрокам (<code>parser/parse_dxf.py</code>)	Готово; джойн перечётной ведомости с деревьями по номеру — нет
2. Применение решений	Вырубить/сохранить/пересадить + расчёт компенсации	Не реализуется: в таблицах датасета нет данных для этого этапа
3. Характеризация + retrieval	Признаки участка (<code>backend/greenplan/site_characterization.py</code>) + поиск похожих проектов (<code>backend/greenplan/pattern_retrieval.py</code>)	Готово — корпус: 14 реальных проектов + 20 синтетических эталонов для типов участков, которых среди реальных нет
4. Назначение паттернов	Разбиение на геометрические зоны (<code>backend/greenplan/zone_partitioning.py</code>); сначала общее решение — стиль участка (регулярный или пейзажный) и ведущий проект-аналог, затем голосование ближайших соседей за приёмы зон в этом стиле (<code>backend/greenplan/pattern_assignment.py</code> , <code>data/pattern_corpus.yaml</code>); единая палитра видов на участок	Готово правилами; опциональное ранжирование LLM — нет
5. Расстановка	Точки/ряды/кластеры через геометрический планировщик (<code>backend/greenplan/deterministic_placement.py</code> , переиспользует <code>backend/core/placement.py</code>); виды — по ассортименту Москвы для типа территории, без инвазивных 369-ПП (<code>backend/greenplan/species_selection.py</code>); газон площадью на свободной земле, новый и сохраняемый (<code>backend/greenplan/lawn.py</code>)	Готово
6. Отчёт решений	Список нарушений и ассортимент (<code>backend/greenplan/violation_report.py</code> / <code>assortment_report.py</code>) + текст-объяснение через YandexGPT 5.1 Pro (<code>backend/greenplan/decision_report.py</code>) + пояснительная записка в DOCX с ведомостью по форме 9 ГОСТ 21.508 (<code>backend/greenplan/document.py</code>)	Готово

`POST /api/greenplan/generate` (быстро, без LLM), `POST /api/greenplan/report` (текст-объяснение через YandexGPT, отдельным запросом) и `POST /api/greenplan/document` (пояснительная записка в DOCX) подключены к фронтенду — подробности в `docs/TECHNICAL_OVERVIEW.md` , раздел 6.3.

Дальше

- Больше реальных проектов в корпусе GreenPlan (сейчас 14 реальных + 20 синтетических)
- Опциональное ранжирование отфильтрованных паттернов LLM
- Параметры генератора (`species/grid_spacing_m/min_tree_spacing_m`) пока настраиваются только через `query string` / Swagger — на фронте нет полей для них, плотность посадки нужно подбирать под масштаб конкретной сцены вручную (см. `backend/README.md` , "Плотность посадки")

Лицензия

Код открыт только для просмотра: копирование, запуск, изменение и распространение без письменного согласия авторов запрещены. Организаторам и экспертам конкурса «Лидеры цифровой трансформации

2026» и ДПиООС разрешено собирать и запускать решение для оценки. Полный текст – [LICENSE](#).

GreenCity — техническая документация проекта

Справочник для того, кто должен разбираться в проекте и отвечать на вопросы по нему (демо/защита).
Описывает состояние ветки `main`. Документация решения по ТЗ (MVP, BPMN, интерпретируемость, библиотеки, проверка DXF, ограничения) — [SOLUTION.md](#), алгоритм GreenPlan подробно — [GREENPLAN_ALGORITHM.md](#).

1. Что это

Сервис автоматического проектирования озеленения города с учётом подземных коммуникаций (хакатон ЛЦТ). Два независимых, но совместимых пути:

1. **Ручной редактор** — импорт DXF, 3D-просмотр, drag&drop объектов с проверкой норм, автогенерация по сетке, правка текстом через LLM, экспорт обратно в DXF.
2. **GreenPlan** — детерминированный конвейер, который предлагает озеленение участка по аналогии с реальными прошлыми проектами (retrieval), а не с нуля и не через ML-обучение.

Оба пути работают на одной и той же модели данных (`Scene`) и одном парсере DXF.

2. Архитектура

DXF-файл → `parser/parse_dxf.py` → `Scene` (JSON) → `backend` (FastAPI) → `frontend` (React + `react-three-fiber`, 3D)

- **backend/** — FastAPI, вся геометрия на `shapely`. Все обработчики асинхронные: тяжёлые расчёты идут в пуле потоков через ограниченную очередь, запросы к LLM — асинхронным клиентом (раздел 2.1). В деве — один процесс с `--reload` (`UVICORN_RELOAD=1`, `UVICORN_WORKERS=1`); в продакшене (`docker-compose.prod.yml`, образ по умолчанию) — `UVICORN_WORKERS` процессов без `--reload`, т.к. GIL не даёт параллелить CPU-геометрию потоками — для нагрузки нужны отдельные процессы. Ответы сжимаются (`GZipMiddleware`). Наружу бэкенд не открыт: к нему обращается только прокси фронтенда. Код разложен по пакетам (карта ниже).
- **frontend/** — React + TypeScript + `react-three-fiber` (3D-сцена); в деве — Vite dev server, в продакшене — статическая сборка за `nginx` (`frontend/Dockerfile.prod`, `/api/` проксируется на `backend`). Страница редактора — `src/pages/editor/` (сама страница с состоянием и обработчиками + панели: топбар, левая панель, Ассистент, GreenPlan, диалог сохранения).
- **parser/** — `parse_dxf.py` (сборка сцены и командная строка) + `dxf_parsing/` (правила слоёв, геометрия, зоны и граница, здания и точечные объекты), общее ядро для CLI и `/api/parse`.
- **converters/** — GeoJSON/SHP/DWG → DXF для внешних источников (Мосгеотрест и т.п.).
- **data/norms/** — нормативы в машиночитаемом виде, обособленно от кода (чтобы юрист/архитектор мог проверить число по первоисточнику, а не по Python).
- **locations/** — корпус GreenPlan: 14 реальных документированных проектов и 20 синтетических эталонов (`<slug>/<slug>.dxf` + `design_rationale.md`) + `location_old/` — синтетические фикстуры для тестов.
- **observability/** — Prometheus + Loki + Promtail + Grafana, поднимаются вместе со всем стекком.

Карта бэкенда

Пакет	Что внутри
main.py	Приложение FastAPI, middleware (лимит тела, журнал запросов, CORS по настройке, gzip), ответ 503 при переполненной очереди, прогрев корпуса при старте, /api/health
api/	Эндпоинты: editor.py (разбор DXF/DWG, каталог, генерация по сетке, правка текстом, экспорт DXF), greenplan.py, accounts.py; protection.py – лимиты частоты и размера тяжёлых эндпоинтов
core/	concurrency.py (тяжёлая работа в пуле потоков, очередь расчётов), schemas.py (сцена), setback_norms.py (отступы), plant_catalog.py, invasive_species.py (369-ПП), placement.py (Placer – допустимая область и нормы), placement_geometry.py (выбор мест, обход препятствий, линии), building_setbacks.py, paths.py, yandex_ai.py (клиент Yandex AI Studio для обеих LLM)
greenplan/	Этапы 3–6: site_characterization, zone_partitioning, pattern_corpus/pattern_retrieval/pattern_library/pattern_assignment, species_selection, deterministic_placement, lawn, violation_report, assortment_report, decision_report (LLM), document (DOCX), dxf_export
text_editor/	Правка текстом: service.py (точка входа), operations.py (словарь операций), prompt.py (контекст модели), llm_client.py (провайдеры), applier.py + ops_placement.py + ops_editing.py + plan_common.py (применение), courtyard_design.py + courtyard_layout.py (дизайн двора)
generation/	Генератор по сетке: greenery_generator.py, natural_sampling.py (Poisson-disk, шум плотности, смесь видов), planting_area.py (где можно сажать)
exchange/	dxf_overlay.py (экспорт поверх исходника), source_store.py (исходные чертежи), export_dxf.py, dwg_batch_converter.py, dwg_job.py, dxf_parser.py (мост к parser/)
accounts/, storage/	Регистрация/вход/JWT, проекты; MongoDB (db.py), Redis (cache.py)
monitoring/	JSON-логи с request_id, метрики Prometheus

Импорты – абсолютные от backend/ (from core.schemas import Scene), пути к данным – только через core/paths.py.

2.1 Асинхронность, очередь расчётов, защита и сеть

Асинхронность. Все эндпоинты – async def, event loop не блокируется ничем тяжёлым:

Работа	Как выполняется
Геометрия и документы (разбор DXF, GreenPlan, экспорт, объяснения, записка, генерация по сетке, контекст и применение плана ассистента)	core/concurrency.run_heavy – пул потоков с тяжёлым слотом
Разбор и экспорт пачки DWG	отдельный процесс через asyncio.create_subprocess_exec; слот пачки (fcntl.flock на весь контейнер) ждётс через asyncio.sleep, поток не занимается
Запросы к LLM (ассистент, текст-обоснование)	openai.AsyncOpenAI: таймаут 120 и 60 с, один повтор; тяжёлый слот не держится
MongoDB, Redis	асинхронные клиенты (AsyncMongoClient, redis.asyncio); bcrypt – в asyncio.to_thread

Работа	Как выполняется
Чтение и запись файлов (загрузки, хранилище источников)	<code>run_light</code> – пул потоков без слота

Правка текстом проходит три шага. Сначала проверяется ключ модели, чтобы без него не считать геометрию. Затем в тяжёлом слоте строится контекст. Потом слот освобождается на время ответа модели и снова занимается для применения плана.

Очередь расчётов (`core/concurrency.py`). Потоки одного процесса делят GIL, и два расчёта параллельно идут не быстрее, а вдвое медленнее каждый. Поэтому на процесс `uvicorn` – `HEAVY_CONCURRENCY` (1) расчёт одновременно, остальные ждут в очереди. Очередь ограничена:

- больше `HEAVY_QUEUE_MAX` (8) ждущих или ожидание дольше `HEAVY_WAIT_S` (120 с) – ответ 503 с `Retry-After`;
- лёгкие запросы (`health`, каталог, вход, проекты) в очереди не стоят;
- замер: во время расчёта `GreenPlan /api/health` отвечает за доли миллисекунды; из 12 одновременных запросов `GreenPlan` 9 выполняются по очереди, 3 получают 503.

Защита тяжёлых эндпоинтов (`api/protection.py`):

Механизм	Что делает
Лимит частоты с одного IP за 10 минут	<code>parse</code> – 30, <code>parse_dwg</code> – 10, <code>greenplan (generate, document, explanations)</code> – 60, <code>llm (edit-with-text, greenplan/report)</code> – 30, <code>export</code> – 30, <code>generate (generate-greenery)</code> – 30. Сверх – 429 с <code>Retry-After</code> . Фиксированное окно в Redis; без Redis лимит не действует (<code>fail open</code>). Все лимиты умножаются на <code>RATE_LIMIT_MULTIPLIER</code> (0 – выключить)
Лимит размера тела (<code>BodySizeLimitMiddleware</code>)	загрузка DXF и DWG – <code>MAX_UPLOAD_MB</code> (200), прочие запросы – <code>MAX_BODY_MB</code> (64). По <code>Content-Length</code> отказ сразу, без него – по мере чтения тела. Ответ 413
Число файлов DWG	не больше <code>MAX_DWG_FILES</code> (60) за загрузку, иначе 400
Уже было раньше	длина просьбы ассистенту – до 2000 символов, история – до 20 правок; <code>k</code> у <code>GreenPlan</code> – от 1 до 9; разбор DWG – не дольше 15 минут

Сеть.

- Наружу опубликованы только фронтенд и Grafana. Бэкенд, MongoDB, Redis, Prometheus и Loki слушают только сеть `compose`.
- Браузер обращается к API по относительным путям `/api/...` на адрес сайта. В дев-режиме их проксирует vite (`frontend/vite.config.ts`, `API_PROXY_TARGET`), в продакшене – nginx (`frontend/nginx/default.conf.template`).
- CORS выключен; при фронтенде на другом домене разрешённые адреса задаются в `CORS_ALLOW_ORIGINS`.
- `uvicorn` запущен с `--proxy-headers`, поэтому IP клиента для лимитов берётся из `X-Forwarded-For` прокси. Доверять заголовку безопасно, потому что к бэкенду никто, кроме прокси, не подключается.
- Swagger (`/docs`, `/openapi.json`) открыт при `API_DOCS=1` (дев, через фронтенд – `http://localhost:5173/docs`), в продакшене выключен.

Docker-compose поднимает 8 сервисов с `healthcheck` на каждом и `depends_on: condition: service_healthy` – бэкенд стартует только когда mongo/redis реально приняли соединение. Код смонтирован `volume`'ами – правки в `backend//parser//frontend/` подхватываются на лету, без пересборки образа (новые зависимости в `requirements.txt` – нет, там нужен `--build`). Для сервера –

`docker-compose.prod.yml` поверх основного файла (`make docker-prod-up`): статика за `nginx` на :80, backend без смонтированного кода и `--reload` (3 процесса, 3 ядра, до 4 ГБ; до 2 пачек DWG одновременно), кеш MongoDB 0,5 ГБ, ротация логов контейнеров; рассчитан на сервер 4 vCPU / 8 ГБ / 40 ГБ SSD (README, «Docker на сервере»).

3. Модель данных: Scene

Единый JSON-контракт (`backend/core/schemas.py`) между парсером, обоими бэкенд-путями и фронтендом:

- `boundary` — граница участка (полигон).
- `restrictions` — зоны ограничений: `type` (`sewer/gas/road/building/pedestrian_path/...`), `severity` (`forbidden/warning/allowed`), `minDistance`, `polygon`.
- `objects` — точечные объекты сцены (деревья/кусты/здания/МАФ), с `type`, `position`, `metadata` (`species`, `generated: true/false`, `provenance` для GreenPlan).
- `windows/canopies/curbs` — декоративная геометрия (не самостоятельные объекты).
- `lawns` — газон GreenPlan (`LawnArea`): полигон с дырками (клумбы кустарника), `area_sqm`, `status` (`new` — устройство газона, `existing` — сохраняемый). Заполняет только GreenPlan; у сцен из `/api/parse` пусто.

`severity: forbidden` — красный, сажать нельзя; `warning` — жёлтый, предупреждение (для ручного редактора — не запрет, для автогенератора — тоже запрет); `allowed` — зелёный, явно разрешённая зона (газон).

4. Парсер DXF (`parser/parse_dxf.py`)

Слои классифицируются **по подстроке в имени** (`POLYGON_RULES`, `POINT_LAYER_RULES`), не по жёсткой карте — имена слоёв не совпадают между проектами. Порядок правил значим (специфичное правило должно стоять раньше общего, иначе общее перехватит совпадение первым).

Ключевые приёмы, оплаченные реальными багами на конвертированных DXF:

- **Слияние по слою, не по сущности** (`_merge_corridors/_merge_polygon_zones`) — на реальных файлах один тип сети/газона может прийти тысячами мелких кусков; без слияния `retrieval/zone_partitioning` считали бы на россыпи вместо одной площади.
- **Вид растения из имени слоя.** Слой с названием вида из справочника («ЛИПА МЕЛКОЛИСТНАЯ») даёт объекту `metadata.species`, бэкенд (`exchange/dxf_parser.py`) по нему подставляет `catalogId` — объект получает 3D-модель и габариты своего вида, а характеристика участка — состав существующих пород.
- **Невалидные контуры чинятся, а не отбрасываются** (`core/shapes.py::polygon_from_points, make_valid`) — одно правило для расстановки, газона, разбиения на зоны, признаков участка, проверки нарушений и генератора по сетке. Раньше часть модулей молча пропускала самопересекающиеся контуры из DXF: на 20_макеева_5 терялось 140 тыс. м² разрешённой земли (GreenPlan не сажал ни одного дерева), а 113 тыс. м² запретных зон не видела проверка нарушений.
- **Коридоры труб/кабелей** — линии одного слоя буферизуются на `minDistance` и объединяются одним вызовом (`quad_segs=2` вместо 8 — экономия вершин на порядок при том же итоге).
- **Обрезка сетей по границе участка** (`_clip_offsite_zones`) — геоподоснова тянет сети далеко за пределы участка; без обрезки на узких вытянутых участках отступы перекрывали его насквозь.
- **Слаботочка vs силовой кабель** — разная серьёзность зоны (`warning` vs `forbidden`) и ширина охранного коридора, проверяется ДО общего правила `ELECTR`, иначе один тип "electrical" перекрывал большую

часть площади из-за смешения проводов связи и питания. Отступ посадки от них при этом одинаковый (2 м дерево, 0,7 м куст – одна строка табл. 9.1 СП 42).

- **Маркер точки ≠ второй объект** – экспорт пишет каждую посадку как POINT + маленький CIRCLE (иначе в CAD её не видно); круг ровно в точке POINT / INSERT того же слоя парсер считает маркером. Без этого повторная загрузка экспортированного плана удваивала все посадки (найденно сквозным тестом).

5. Ручной редактор (backend)

Эндпоинт	Что делает
POST /api/parse	DXF → Scene
POST /api/parse-dwg	Папка из нескольких .dwg → Scene : батч-конвертация через dwg2dxf /LibreDWG + слияние в один документ (backend/exchange/dwg_batch_converter.py),дальше тот же parse_dxf.py . Файлы, которые не удалось сконвертировать, не роняют запрос – попадают в Scene.dwgConversionWarnings . Разбор – в отдельном процессе: файлы пачки параллельно в DWG_WORKERS процессах (каждый конвертирует свой файл и оставляет только слои, которые читает парсер),до DWG_MAX_PARALLEL пачек с разных аккаунтов одновременно (exchange/dwg_job.py , dwg_batch_converter.py); реальная пачка из 5 файлов – 19 с вместо 28,5, три одновременные – 25/26/46 с вместо 27/58/87 с.
POST /api/generate-greenery	Деревья/кусты/газон по сетке (без ML), см. ниже
POST /api/edit-with-text	Правка плана текстом через LLM
POST /api/export-dxf	Scene → DXF: слои результата поверх исходного чертежа (раздел 9)
GET /api/catalog	Каталог видов/МАФ (кэш в Redis 30с)

5.1 /api/generate-greenery (generation/)

Детерминированный генератор: кандидаты деревьев – Poisson-disk разброс с радиусом grid_spacing_m (без рядов сетки, с фиксированным сидом), жадный отбор по min_tree_spacing_m, пятна гуще и реже посадки и смесь видов (natural_sampling.py); всё внутри allowed -зон минус запретные (расширенные на норматив отступа для вида посадки). Кусты – регулярная сетка, но её узлы – **центры групп** (по 3 куста вокруг, п.17 ТЗ "группировка"), с ограничением сверху (MAX_GENERATED_BUSH_CLUSTERS=60) и равномерным разбросом (pick_spread, k-средние), иначе двор "закатывается" кустами сплошным ковром. Газон – плитки без отступа по виду посадки (у дёрна нет корней/кроны).

Плотность – три рычага сразу: grid_spacing_m, размер зоны газона в конкретной сцене, min_tree_spacing_m. Универсального дефолта нет – параметр специально вынесен наружу под конкретную сцену.

5.2 Правка текстом (`text_editor/service.py`) — ключевой архитектурный принцип: LLM не считает координаты

Два шага:

1. LLM получает JSON-контекст (`text_editor/prompt.py` : контур участка, здания, ориентиры, кандидатные ряды, свободные области, выделенные пользователем зоны, каталог) и системный промпт, возвращает список операций (`text_editor/operations.py` : `add / remove / move / rotate / place_along / place_in_area / cover_area / connect / define_zone / design_area / run_greenplan / ...` — 21 тип) + текстовое объяснение.
2. Детерминированный `PlanApplier` (`text_editor/applier.py` + `ops_placement.py` + `ops_editing.py`, поверх `core.placement.Placer`) выполняет каждую операцию с учётом отступов по породе: ищет ближайшую допустимую точку, раскладывает вдоль контура, строит маршрут в обход препятствий (граф видимости + Дейкстра для `connect`). Если операция невыполнима — не исключение, а запись в `rejected` с человекочитаемой причиной; одна сломанная операция от модели не роняет весь план.

Каталог в контексте: базовые позиции (МАФ, газон, мощение) — все, виды — выборкой по классу формы (сначала базовый ассортимент 515-ПП), всегда с русским названием. Виды, названные в просьбе («липы», «клёны», «туи»), попадают в контекст сверх выборки: `prompt.mentioned_species` сравнивает слова просьбы с названиями видов с учётом окончаний. Без этого модель не находила липу мелколистную среди 220 видов и сажала что-то похожее по размеру.

История чата: фронтенд отправляет прошлые правки (`history` : просьба, ответ, что сделано), модель получает последние четыре в сокращённом виде. Так понимаются отсылки «убери их», «то же самое у второго дома». Отменённые правки в историю не попадают.

`run_greenplan` — мост к GreenPlan: на «озелени участок в регулярном стиле, побольше лип» модель выбирает параметры GreenPlan (стиль, что сажать, предпочтительные виды, дорожки, освещение, скамейки). Бэкенд проверяет виды (только `species_*` своей категории) и возвращает параметры в `greenplan` ответа, а фронтенд запускает настоящий GreenPlan на сцене ответа — с его панелью решений, нарушениями и запиской.

Последнюю правку ассистента можно отменить кнопкой в чате, пока план после неё не менялся (сцена до и после хранится на фронтенде; ручная правка после неё отмену убирает, чтобы не стереть её молча).

Отбор существующих объектов (`remove_where`, `replace_where`, `thin_out`, `resize`, `face`, `align_along`, `set_count`) кроме места (у цели, в радиусе от точки) фильтрует по виду (`species` : «Клен» — любой клён, «Клен остролистный» — только он; падеж и ё не важны, `plan_common.name_matches` ; у объектов без вида — по русскому названию типа: «шезлонг», «урна») и по id (`ids`). В историю чата уходят id объектов, созданных каждой правкой, — «убери их», «сделай их поменьше» трогают ровно их, а не всё в радиусе от места посадки. Подъезды в контексте — с номером («подъезд 1») и id, чтобы на них можно было сослаться. Урны из чертежа (тип `urn`) и из каталога (`trash`) — один тип. Правки без удаления (масштаб, замена, разворот), не нашедшие объектов у точки, расширяют круг до ближайших и пишут об этом; удаление так не делает.

Честность ответа: поля, которые операция не поддерживает, не отбрасываются молча, а попадают в предупреждения; если не выполнено ничего, ответ модели («площадка огорожена») заменяется на «Не получилось: ...» с причиной; под ответом в чате — что реально сделал планировщик.

Оценка на настоящей модели — `tools/ai_editor_eval/` (`.venv/bin/python tools/ai_editor_eval/run.py [id ...]`): 29 реалистичных просьб на трёх участках (пустой двор, двор с

детской площадкой, реальный проект на 600+ объектов), включая цепочки с отсылками («посади 4 берёзы...» → «а теперь убери их»), вопросы, опечатки и несуществующий вид. Каждая проверяется объективно по сцене (что добавлено/удалено, какие виды, где, нарушения норм у новых объектов). Каждый случай – платный запрос (~11–28 тыс. токенов), в тесты не входит; найденные провалы воспроизведены офлайн в `backend/tests/text_editor/test_editor_polish.py`. Результат доводки: 14/24 → 29/29.

`LLM_PROVIDER` – `yandex` (по умолчанию: Yandex AI Studio, `core/yandex_ai.py`, модель `YANDEX_CLOUD_MODEL` – по умолчанию Qwen3 235B A22B: строгий JSON по длинной инструкции и без рассуждений, которые съедают лимит ответа) или `gemini`, оба через OpenAI-совместимый клиент. Без ключа – `LlmNotConfiguredError` → 503, а не падение сервиса.

`design_area` – самый сложный оператор: `CourtyardDesigner` (`text_editor/courtyard_design.py`, выбор области и каркас дорожек – `courtyard_layout.py`) собирает целый двор одной операцией в порядке "от каркаса к деталям": выбор области (огорожена зданиями) → скелет дорожек (MST между входами) → мощение → фонтан/клумбы/фонари/лавки/изгородь/деревья вразброс/кусты вдоль дорожек. Виды, которые модель не назвала сама (`tree_ids`/`bush_ids`), подбирает `species_selection.courtyard_palette()` – тот же подбор, что у GreenPlan, для дворовой территории: три дерева разных родов из ассортимента Москвы с приоритетом базового ассортимента 515-ПП и вид для изгороди вдоль дорожек (без колючих и чувствительных к реагентам).

5.3 Отступы (`setback_norms.py`, СП 42.13330.2016 табл. 9.1 / 743-ПП табл. 3.6.1)

Зона	Дерево	Куст
building	5.0 м	1.5 м
road	2.0 м	1.0 м
electrical, signal_cable	2.0 м	0.7 м
heat_network	2.0 м	1.0 м
sewer/gas	1.5 м	0
water_pipeline	2.0 м	0
pedestrian_path	0.7 м	0.5 м

Плюс правила по породе (`SPECIES_SETBACK_RULES`), которые только ужесточают таблицу: липа/клён/дуб/каштан/тополь – 10 м от здания (743-ПП, прим. 3 к табл. 3.6.1), колючие (роза, барбарис, боярышник) – 2 м от дорожек и детских площадок (СП 82.13330.2016, п. 9.22), породы, чувствительные к прогреву, – 2–4 м от теплотрассы (МГСН 1.02-02, п. 4.2.8). Применяются везде, где ставятся или проверяются посадки: генератор по сетке, `Placer` (правка текстом, `design_area`, GreenPlan), `violation_report.py`, проверка при перетаскивании на фронте. Портитован 1:1 в `frontend/src/setbackNorms.ts` (ручная синхронизация); совпадение таблицы с `data/norms/sp-42-13330-2016/setbacks.csv` проверяет автотест. Выписки из всех актов перечня – `data/norms/README.md`.

6. GreenPlan – 7 этапов

Этап	Модуль	Статус
0. Справочники	<code>data/norms/</code>	Готово: 10 актов, отступы и правила по породе в коде, инвазивные виды исключены из каталога
1. Импорт DWG→DXF	<code>parser/parse_dxf.py</code>	Готово; джойн ведомости с деревьями по номеру – нет

Этап	Модуль	Статус
2. Решения по существующим деревьям + компенсация	—	Не реализуется: в таблицах датасета нет данных для этого этапа
3. Характеризация + retrieval	<code>site_characterization.py</code> , <code>pattern_retrieval.py</code>	Готово, корпус — 14 реальных проектов + 20 синтетических эталонов
4. Назначение паттернов	<code>zone_partitioning.py</code> , <code>pattern_assignment.py</code>	Готово правилами; ранжирование LLM — нет
5. Расстановка	<code>deterministic_placement.py</code> (поверх <code>core/placement.py</code>), виды — <code>species_selection.py</code> , газон — <code>lawn.py</code>	Готово
6. Отчёт решений	<code>violation_report.py</code> , <code>assortment_report.py</code> , <code>decision_report.py</code> , <code>greenplan/document.py</code>	Готово: нарушения + ассортимент + текст через YandexGPT 5.1 Pro + пояснительная записка в DOCX

Принцип на всех этапах: **LLM не считает координаты и не придумывает паттерн** — только пересказывает готовые решения текстом. Алгоритм подробно — [GREENPLAN_ALGORITHM.md](#).

6.1 Конвейер по шагам

- `site_characterization.characterize_site(scene)` → `SiteCharacteristics` (площадь, доля пригодной площади, состав по типам зон, тип территории по правилам: двор / улица / площадь / парк-сквер / промышленная-охранная / не определён — только «не определён» помечен `territory_type_is_heuristic=True`).
- `pattern_retrieval.nearest_projects()` → проекты корпуса (34), упорядоченные по косинусному сходству z-нормированного вектора признаков (алгоритм подробно — раздел 6.2 ниже).
- `zone_partitioning.partition_zones(scene)` → геометрические зоны участка: `building_border` / `path_corridor` / `site_edge` / `open_area`, по приоритету захвата площади.
- `pattern_assignment.assign_patterns()` → решение в два уровня: сначала общее на участок, потом по зонам — чтобы у участка был единый стиль.

Общее решение на участок. У каждого приёма есть стиль (`PatternSpec.style`): - регулярный — диагональные ряды, боскет, треугольная сетка, концентрические кольца; - пейзажный — волнистые ряды, рожи, свободный разброс; - нейтральный — изгородь вдоль дорожки и кольцо у здания (бывают в любом стиле), заливка без замысла.

Стиль участка выбирают k самых похожих проектов (сходство не ниже `MIN_VOTE_SIMILARITY = 0.2`), у которых для видов зон этого участка есть решение регулярного или пейзажного стиля; голос весит столько, сколько сходство. Ведущий проект — самый похожий проект победившего стиля. Стиль и ведущий проект записываются в каждое решение (`site_style`, `lead_project`) и выводятся первыми в записке, на панели GreenPlan и в тексте-обосновании.

Проект только с нейтральными решениями в голосовании за стиль не участвует, и стиль решают следующие по сходству. Поэтому у уличных эталонов, ближайших к типовым дворам, во дворах есть приёмы определённого стиля: боскет у 31, сад по треугольной сетке у 35. Без них стиль решали дальние, в основном пейзажные проекты: из 14 реальных проектов определённый стиль есть у семи, и

только один из них регулярный. В итоге все шесть тестовых участков выходили пейзажными, теперь их 3 регулярных и 3 пейзажных.

Зачем: проекты корпуса описаны частично — в основном одним-двумя видами зон. Без общего шага каждый вид зоны голосовал сам по себе, и участок собирался из приёмов трёх-четырёх проектов разного стиля: в одном дворе волны, диагональные ряды и заливка одновременно.

Решения по зонам — голосование отдельно по каждому виду зоны: - **кто голосует**: к самым похожих проектов, у которых есть решение для этого вида зоны в стиле участка или нейтральное (проект без такой записи место избирателя не занимает), со сходством не ниже `MIN_VOTE_SIMILARITY`; заливка без замысла (`generic_fill`) голосует, только если других решений нет; - **вес голоса** — сходство проекта с участком; - **раздача зон**: зоны одного вида делятся между паттернами избирателей по площади, пропорционально весу голосов: крупнейшая зона — самому весоному паттерну, каждая следующая — паттерну, сильнее всех недобравшему свою долю площади. Если избиратели согласны, все зоны вида получают один приём; раньше один победитель забирал все зоны вида, и участок заливался одним приёмом; - **источник**: `source_project` — самый похожий проект среди голосовавших за паттерн зоны, `confidence` — доля избирателей, голосовавших за этот паттерн; - **нет избирателей** — типовой приём стиля участка (`pattern_library.default_pattern`: для открытых зон регулярного участка — треугольная сетка, пейзажного — рощи) с `confidence=0`, `source_project=None`; в записке — «типовое решение в стиле участка — проверить». 5. `deterministic_placement.generate_for_scene()` → геометрия по 3 семействам паттернов: `linear` (ряд вдоль центральной линии), `clustered` (рощи/группы), `area_fill` (Пуассоновская заливка/сетка) — один параметризуемый алгоритм на семейство, не на паттерн. Проверка отступов **во время** расстановки через `Placer.is_free`, не постфактум; кандидаты площадной заливки и рощ заранее отсеиваются от занятых существующими объектами точек, иначе на участке с сотнями старых деревьев центр рощи попадал к старому дереву и вся группа отклонялась. Виды растений подбирает `species_selection.select_site_species()` — единая палитра на участок: пары (вид зоны, паттерн) идут по убыванию площади, крупнейшие задают палитру, следующие берут из неё виды своей формы и добавляют новые, только пока в палитре меньше 4 деревьев и 4 кустарников (предел мягкий: роль, которой по форме не подходит ни один вид палитры, получает свой). Фильтры для каждого вида: только виды из ассортимента для озеленения Москвы с «+» для типа территории участка, без инвазивных (369-ПП), без кодов ассортимента, запрещающих место (самосев; у детских площадок; у дорожек — чувствительные к реагентам), без колючих вдоль дорожек (СП 82 п. 9.22), без широкой кроны в полосе у здания (743-ПП). Форма — по морфологической группе вида под роль паттерна (изгородь и аллея — один вид, роща и свободная посадка — смесь разных родов); приоритет — базовый ассортимент 515-ПП для дворов, затем основной ассортимент. Выбор и его основание попадают в `ZoneAssignment` (`tree_species/bush_species/species_basis`) и в текст отчёта. Объём ограничен: не больше 400 деревьев и 400 кустов на зону (`MAX_OBJECTS_PER_ZONE`; у рядов — за счёт шага, чтобы ряды оставались по всей зоне) и не больше 10 000 новых посадок на участок (`MAX_NEW_OBJECTS_PER_SITE`; сверх — равномерное прореживание с сохранением доли каждой зоны). Без этого крупные участки давали десятки тысяч объектов (02_peschany — 54 тыс., 05_klykova — 54 тыс.). 6. `lawn.plan_lawns()` → газон (раздел 6.5): площадь, а не объекты. 7. `violation_report.find_violations()` → независимая проверка всех объектов сцены против всех зон (STRtree для отсеивания кандидатов, точная геометрия для итогового решения — иначе на крупных участках O(объекты*зоны) уходил в 13+ секунд). 8. `assortment_report.summarize_assortment()` + `lawn_assortment()` → сводка по видам/категориям только сгенерированных объектов (шт.) и строка нового газона (м²). 9. `decision_report.generate_report()` → связный текст-объяснение через YandexGPT 5.1 Pro в Yandex AI Studio (Responses API; тот же ключ и каталог, что у ИИ-ассистента,

модель — `YANDEX_CLOUD_REPORT_MODEL`) по уже посчитанным решениям — LLM пересказывает факты человеческим языком, не считает и не придумывает.

6.2 Как работает поиск похожих проектов (`backend/greenplan/pattern_retrieval.py`)

Не ML и не обучение с градиентом — k ближайших соседей по косинусному сходству вектора числовых признаков. Это и есть "обучение" при малом N (34 проекта): не выдумываем решение с нуля, а смотрим, что реально делали архитекторы на похожих участках.

1. **Вектор признаков** (`_raw_vector`) собирается из `SiteCharacteristics`: `plantable_ratio` (доля пригодной площади), `log1p(площадь участка)` (лог — иначе огромный разброс площадей задавил бы остальные признаки), `building_count`, доли каждого типа зоны ограничений от фиксированного списка `KNOWN_ZONE_TYPES` (`building/road/pedestrian_path/gas_pipeline/sewer/water_pipeline/electrical/playground_zone` — только эти 8, зона другого типа в вектор не попадает и не искажает доли известных), `one-hot territory_type` (шесть категорий) — добавляется, только если `territory_type_is_heuristic == False`, то есть для любого определённого типа территории; «не определён» в вектор не попадает.
2. **Стандартизация (z-score)** по статистике самого корпуса (`_corpus_matrix`, среднее/std считаются один раз и кэшируются `lru_cache`) — без неё `log1p(площадь)` (порядок 5–12) забил бы `plantable_ratio` (порядок 0–1) в косинусном расстоянии почти до нуля. Стандартизуются и корпус, и вектор нового участка одними и теми же `mean/std` корпуса.
3. **Косинусное сходство** между стандартизованным вектором участка и каждым проектом корпуса (`nearest_projects()`); признак, постоянный по всему корпусу (`std=0`), не делится на ноль (заменяется на 1.0).
4. Возвращаются `k` (по умолчанию 3) проектов с наибольшим сходством — список `NeighborMatch(slug, similarity)`, по убыванию.

Дальше ближайшие проекты идут в `pattern_assignment.py` как избиратели за паттерн для каждого вида зоны (голосование, взвешенное по `similarity`, с порогом сходства, см. п.4 конвейера выше) — само сходство участков не выбирает паттерн напрямую, только формирует, кто и с каким весом голосует.

Проверка корректности: каждый из 34 проектов корпуса, поданный как "новый участок", должен находить сам себя ближайшим соседом с `similarity = 1.0` — это базовый регрессионный тест retrieval (если он ломается — либо вектор признаков считается по-разному для корпуса и запроса, либо стандартизация рассинхронизировалась).

6.3 API

`POST /api/greenplan/generate` — быстрый путь (секунды, без LLM): тело `{scene, options}` (параметры — раздел 6.6), ответ — сцена с результатом, решения по зонам, нарушения, ведомость посадок, ведомость благоустройства (`improvements`) и замечания к параметрам (`notes`). Повторный запуск заменяет прошлый результат GreenPlan в присланной сцене, а не добавляет второй слой (`greenplan/pipeline.py`). `POST /api/greenplan/report` — отдельно текст-объяснение (YandexGPT, секунды) — разнесены специально, чтобы фронт не ждал текст ради уже готовой расстановки. Расстановка идёт через очередь расчётов (раздел 2.1): конкурентные тяжёлые запросы под одним GIL-процессом не параллелятся, а тормозят друг друга (~11х). Текст-обоснование — асинхронный запрос к модели, очередь не занимает.

`POST /api/greenplan/explanations?format=json|csv` — объяснение каждой посадки со ссылкой на НПА и пункт, отклонённые кандидаты и зоны запрета (раздел 9.1).

`POST /api/greenplan/document` — пояснительная записка в DOCX (`greenplan/document.py`) по текущей сцене и решениям GreenPlan — «сценарий выгрузки документации» из ТЗ. Разделы: сведения об участке, нормативная база и применённые отступы (в т.ч. по породе), принятые решения с происхождением каждого («по аналогии с проектом X» / «типовое решение — проверить»), проекты-аналоги с цитатами, ведомость элементов озеленения по форме 9 ГОСТ 21.508-2020, проверка норм (нарушения новых посадок — поимённо, существующих — сводкой по видам ограничений), технические требования к посадке по СП 82.13330.2016, ограничения и допущения; текст от LLM — только приложением с пометкой «текст ИИ — проверить». Нарушения, ведомость и характеристики пересчитываются на сервере по присланной сцене. Кнопка на панели GreenPlan.

6.4 Retrieval-корпус (`data/pattern_corpus.yaml` , `backend/greenplan/pattern_corpus.py`)

34 проекта: 14 реальных (реальный DXF + `design_rationale.md` с цитатой из реальной пояснительной записки, откуда взят паттерн) и 20 синтетических эталонов `21_..._primer` — `30_..._primer` — законченные образцы приёмов, которых среди реальных нет или мало: шумозащитная полоса вдоль дороги, круглая площадь с клумбой и лучами, двор с детской площадкой, бульвар, дом в строгом обрамлении, сад диагональных полос (одна полоса — один вид), волновой сад, регулярный сад квадратов с дубами, озеленённая парковка, сквер над инженерными сетями; и 10 фрагментов улиц с жилыми домами и подъездами (`31_...` — `40_...`): улица с палисадниками, с магазинами на первых этажах, угловой квартал, дом с двором за ним, таунхаусы, поликлиника со сквером у входа, длинный дом вдоль проезда, улица с разделительной полосой, с велодорожкой, тупик с разворотной площадкой. Их собирает генератор `tools/primers/build_primers.py` (`make primers`; уличные — `street_primers.py`): дорожки, сети, МАФ и посадки конкретных видов (вид — в имени слоя DXF), и каждая посадка и МАФ проверяются по тем же нормам, что и GreenPlan, — нарушений в эталонах нет (тест `test_primers.py`). `design_rationale.md` каждого — описание замысла, помеченное как синтетическое; его пересказ — `source_quote` в `data/pattern_corpus.yaml`, из которого LLM пишет «по аналогии с проектом X». Поиск сравнивает по геометрии участка, а не по тому, реальный проект или нет. Самопроверка: каждый проект корпуса находит сам себя как ближайшего соседа (`similarity=1.0`) — базовый тест `retrieval`.

Признаки проектов корпуса посчитаны заранее и лежат в `data/pattern_corpus_features.json` вместе с `sha256` исходного DXF. Раньше первый запрос GreenPlan в каждом новом процессе разбирал все DXF корпуса (~25 с). Теперь это чтение файла; проект, чей DXF изменился, пересчитывается на лету с предупреждением в лог. После правки DXF корпуса или `characterize_site` файл обновляется командой `make corpus-features`; устаревший файл ловит тест `test_features_file_is_up_to_date`.

6.5 Газон (`backend/greenplan/lawn.py`)

«Травянистые покрытия» из ТЗ. Газон в проекте считают площадью (ведомость по форме 9 ГОСТ 21.508: «Газон — 1240 м²»), поэтому GreenPlan не раскладывает плитки, как генератор по сетке, а строит полигоны после расстановки посадок:

- **где:** разрешённые зоны (слои GRASS/LAWN/ГАЗОН и вычисленная парсером открытая земля; если их нет — весь участок) в границе участка, минус твёрдые покрытия и сооружения (здания, проезды, дорожки, площадки, прочие покрытия, запретные охраняемые зоны), минус клумбы кустарника (круг кроны, соседние кусты ряда сливаются в одну клумбу);
- **над сетями — можно:** таблица 9.1 СП 42.13330 и 3.6.1 743-ПП нормируют расстояния только для деревьев и кустарников. Деревья стоят в газоне; приствольные лунки не вычитаются;
- **очистка:** полосы уже 0,5 м и куски меньше 5 м² отбрасываются, контур упрощается с точностью 0,25 м;
- **статус:** часть на существующем покрытии GRASS/LAWN/ГАЗОН — `existing` (сохраняется), остальное — `new`. В ведомость, объём посева и DXF идёт только новый газон;

- **вид:** «Газон обыкновенный», посев из устойчивой травосмеси (ППМ 515-ПП, табл. 4); в записке — нормы высева (СП 82 п. 9.24) и полив 10 л/м² (п. 9.27).

Газон — производная от посадок: записка (`/api/greenplan/document`) и экспорт GreenPlan пересчитывают его по присланной сцене, а не берут из запроса, поэтому ручные правки после GreenPlan учитываются. На 3D-сцене газон рисует `frontend/src/scene/Lawns.tsx` (новый — светлее, существующий — темнее); на экране он обновляется при следующем запуске GreenPlan.

6.6 Параметры пользователя (`greenplan/options.py`, `greenplan/improvements.py`, `greenplan/pipeline.py`)

Кнопка GreenPlan открывает диалог параметров; без изменений — поведение по умолчанию (стиль по аналогам, деревья, кустарники и газон, без благоустройства). Последние параметры запоминаются в браузере.

- **Стиль участка:** авто (голосование аналогов), регулярный или пейзажный. Заданный стиль заменяет голосование; ведущий аналог — самый похожий проект этого стиля, если такого нет — зоны получают типовые приёмы стиля.
- **Что сажать:** деревья, кустарники, газон — каждое можно выключить.
- **Предпочтительные виды** деревьев и кустарников: ставятся первыми в любой роли своей категории (ограничение по форме для них снято) и входят в палитру участка. Нормы действуют как для всех: ассортимент для типа территории, 369-ПП, коды ограничений, колючие у дорожек, широкая крона у здания. Почему вид не использован — в `notes` и на панели.
- **Благоустройство** — до посадок, чтобы посадки соблюдали отступы от него:
 - дорожки: каркас дизайна двора (от подъезда к подъезду, к парковке) в дворах участка, до трёх; заводятся зонами `pedestrian_path` шириной 1,5 м (слой `NEW_PATHS` в DXF) — отсюда нормативный отступ посадок (0,7 м для дерева), газон их обходит, вдоль них появляются полосы под изгородь;
 - освещение: фонари через 20 м зигзагом вдоль новых дорожек и снаружи вдоль существующих, не ближе 4 м к деревьям (СП 42, табл. 9.1); в охранные зоны сетей опоры не ставятся — фонарь ищет место на другой стороне или со сдвигом до 8 м, остаток честно отмечается в `notes`;
 - скамейки с урнами: через 30 м вдоль новых дорожек.

Пояснительная записка получает параметры и замечания (раздел 3) и ведомость благоустройства (раздел 4.1); газон, выключенный в параметрах, в записку не попадает.

7. Каталог видов и МАФ (`core/plant_catalog.py`)

`CatalogItem`: семантика (`category`, `setback_kind`, `dimensions`) для генератора + внешний вид (`model`, `render`) для фронта. Есть `.glb` — рисуется модель; нет — примитив-заглушка по габаритам (штатный режим, не `bar`). `catalog_generated.json` — записи из реальных паков моделей (`tools/convert_models.mjs`), дополняет статический каталог, не заменяет. Виды из перечня инвазивных ППМ 369-ПП (`core/invasive_species.py`, 32 вида) из каталога исключаются при загрузке — их не предложит ни один путь.

8. Аутентификация и проекты

JWT access (15 мин) + refresh (30 дней, сессия в Redis — реальный отзыв при logout). MongoDB — только `users/projects` (постоянное состояние); Redis — то, что не жалко потерять (rate-limit, кэш каталога, сессии).

Fail-open: недоступен Redis — лимиты/сессии не блокируют работу; недоступна MongoDB — только

/api/projects /api/auth/* отвечают 503, сам редактор (DXF/генерация/правка текстом) не зависит от БД вовсе. Лимит – 3 проекта на пользователя.

9. Экспорт DXF (exchange/dxf_overlay.py, exchange/export_dxf.py)

ТЗ: исходные слои подосновы, коммуникаций и городской среды не изменяются и не перезаписываются, результат – только на отдельных слоях. Поэтому экспорт дописывает результат **поверх исходного чертежа**, а не собирает DXF из сцены.

- **Хранилище исходников** (exchange/source_store.py, каталог GREENCITY_SOURCES_DIR, в Docker – том sources_data): при загрузке DXF сохраняется сам файл, папка DWG – сами .dwg в порядке загрузки; хранение по sha256, одинаковый файл лежит один раз. В сцене остаётся только meta.sourceId.
- **Наложение** (overlay_scene): исходник заново разбирается тем же парсером (база), сцена сравнивается с базой по id объектов и зон, и в копию исходника дописываются только отличия – в его координатах и единицах ($X = x / scale + origin.x$, $Y = z / scale + origin.y$, обратное к parser Transform). Исходные сущности не трогаются: подписи, штриховки, блоки, размеры и нераспознанные слои остаются как были.
- **Слои результата:** посадка и благоустройство GreenPlan – NEW_TREE, NEW_BUSH, NEW_LAWN (заливка), NEW_PATHS, NEW_LAMP ...; правки пользователя (ИИ-ассистент, панель «Добавить объект», новое место сдвинутого исходного объекта) – USER_*; места исходных объектов, которые пользователь удалил или сдвинул, – крестик на USER_REMOVED (сам объект в своём слое остаётся); зоны, выделенные в редакторе, – USER_ZONES.
- **XDATA GREENCITY** на каждой сущности результата: id объекта, вид, происхождение («GreenPlan: приём, аналог» или «правка пользователя»). По id посадка находится в файле объяснений (раздел 9.1).
- **Пачка DWG** при экспорте заново конвертируется тем же конвейером, что и при загрузке, в отдельном процессе (dwg_job.run_export): загрузка не платит за сохранение собранного чертежа, платит только экспорт (реальная пачка Харьковской – 18 с).
- **Без исходника** (проект сохранён до появления хранилища, файл удалён, чертёж формата R12) экспорт собирает DXF из сцены (export_dxf.scene_to_dxf, в метрах от центра участка, со слоями NEW_*/USER_*), ответ помечен заголовком X-GreenCity-Export: rebuilt, редактор показывает предупреждение.

Сборка из сцены – зеркало парсера: экспортированный план загружается обратно тем же парсером без потери и удвоения посадок. Объекты с width+depth (мощение/газон/изгородь) – реальным прямоугольником. Severity зон кодируется цветом (AutoCAD Color Index) + XDATA.

9.1 Объяснение каждой посадки (greenplan/explanations.py, POST /api/greenplan/explanations)

ТЗ, п. 8 и 7.2.6: для каждой посадки – объяснение со ссылкой на нормативный акт и пункт, выгружаемое вместе с планом. Кнопки «Объяснения посадок: JSON / CSV» на панели GreenPlan.

- **Каждая новая посадка** (GreenPlan и правки пользователя): вид, координаты в метрах сцены и в координатах исходного чертежа, слой DXF, место на участке и приём, проект-аналог и стиль участка, основание подбора вида (ассортимент Москвы по типу территории, ППМ 515-ПП, без инвазивных ППМ 369-ПП) и до пяти ближайших ограничений – фактическое расстояние, требуемый отступ и пункт НПА, который его задаёт (setback_norms.setback_basis: СП 42.13330.2016 табл. 9.1 / 743-ПП табл. 3.6.1, правила по породе – 743-ПП прим. 3, МГСН 1.02-02 п. 4.2.8, СП 82.13330.2016 п. 9.22; опора освещения – СП 42 табл. 9.1). Зазор между соседними объектами – правило планировщика, так и помечено, а не выдаётся за норму.

- **Отклонённые кандидаты:** точки, которые расстановка рассматривала, но не поставила по норме (порода у здания или теплосети, опора освещения, колючие у дорожек), – с причиной и пунктом; собирает `RejectionLog` во время расстановки, подробно до 300 на участок, остальные – счётчиком по правилам.
- **Зоны запрета посадки:** каждое ограничение участка и нормативная полоса вокруг него для дерева и кустарника с пунктом НПА – туда расстановка точки не рассматривает вовсе.
- JSON – полная структура; CSV (разделитель «;», UTF-8 с BOM для Excel) – строка на посадку с ближайшим нормативным ограничением, на отклонённый кандидат и на зону запрета.

10. Тесты

Около 1080 тестов (951 в `backend/`, 125 в `parser/`), прогон около минуты (`pytest -n auto`). Лежат по тем же пакетам, что и код (`backend/tests/core/`, `greenplan/`, `text_editor/`,...; `parser/tests/`).

- **Юнит-тесты** на двух семействах фикстур: синтетические (`backend/tests/helpers.py`) – точная геометрия для точных проверок; реальные – `locations/location_old/` и корпус `locations/*` для проверки «расстановка реально не нарушает нормы» на настоящих данных. Независимые проверки норм в тестах считают расстояния сами (через `STRtree`), а не тем же кодом, что расставлял.
- **Сквозные тесты** (`backend/tests/e2e/`, `make test-e2e`) – настоящее приложение через HTTP на реальных DXF: загрузка → GreenPlan → текст-обоснование → пояснительная записка → экспорт DXF → повторная загрузка (три разных участка); загрузка → правка текстом → генерация → экспорт; регистрация → проект → сохранение результата GreenPlan → обновление токена → выход. Подменены только внешние границы: MongoDB/Redis (`mongomock_motor/ fakeredis`) и клиенты LLM (фиксированные ответы). Ключи LLM из окружения стираются перед каждым тестом – ни один тест не делает платный запрос.
- **Асинхронность и защита** – `tests/core/test_concurrency.py` (пул потоков, очередь расчётов, отказ при переполнении и по таймауту), `tests/api/test_protection.py` (429 по IP и группам, 413 по `Content-Length` и при потоковой передаче, лимит файлов DWG, 503 с `Retry-After`, закрытый CORS).
- **Сверка с нормативами:** таблица отступов в коде сверяется с `data/norms/sp-42-13330-2016/setbacks.csv`, список конфликтов ассортимента с 369-ПП – с CSV ассортимента.

CI (GitHub Actions и SourceCraft): `ruff`, `oxlint`, проверка типов TypeScript, все тесты с покрытием, сборка и запуск Docker-стека.

11. Наблюдаемость

Prometheus (HTTP + бизнес-метрики: парсинги/генерации/экспорты по типу, исходы LLM-правки и GreenPlan-отчёта, регистрации/входы, блокировки лимитов частоты по группам, отказы 503 из-за очереди расчётов и 413 из-за размера запроса) + Loki/Promtail (JSON-логи с `request_id`) + готовые дашборды Grafana. Всё поднимается одной командой вместе с основным стеком, деградация observability не роняет сам сервис.

12. Частые вопросы (для защиты)

Почему не ML/нейросеть? Датасет – единицы-десятки реальных проектов, для полноценного обучения этого недостаточно и негде проверить обобщение. Вместо этого – `retrieval` (поиск похожих реальных решений) + детерминированная геометрия; LLM – только для пересказа готовых решений человеческим языком и для понимания просьб в правке текстом (операции, а не координаты), никогда не для координат/паттернов с нуля.

Что будет под нагрузкой? Обработчики асинхронные, тяжёлые расчёты идут по одному на процесс uvicorn через ограниченную очередь, запросы к LLM не занимают ни поток, ни очередь. Переполненная очередь даёт 503 с `Retry-After`, а не зависший запрос; частые запросы с одного адреса — 429. Сервер при этом продолжает отвечать на лёгкие запросы. Пропускная способность растёт числом процессов (`UVICORN_WORKERS`), подробности — раздел 2.1.

Можно ли обратиться к API в обход сайта? Бэкенд наружу не опубликован, к нему ходит только прокси фронтенда. Через адрес сайта API доступен, как и нужно браузеру, но с лимитами частоты и размера. CORS выключен, Swagger в продакшене выключен.

Что если LLM недоступна? Текстовая правка отвечает понятной ошибкой (503), GreenPlan отчёт — `report=null` с `report_error`, сама расстановка (детерминированная часть) не страдает, пояснительная записка формируется без приложения с текстом ИИ.

Почему нет решений по существующим деревьям и компенсации (Этап 2)? Для него нужна перечётная ведомость и справочники ценности видов, которых в таблицах датасета нет. Этап не имитируется; в пояснительной записке это указано в разделе «Ограничения и допущения».

Считаете ли вы газон? Да, площадью: GreenPlan строит газон на свободной от покрытий и клумб земле, делит его на новый и сохраняемый, новый идёт в ведомость в м², в DXF (слой `NEW_LAWN`) и в записку с нормами посева и полива. Над инженерными сетями газон допустим — нормы отступов касаются только деревьев и кустарников.

Откуда берутся виды растений? Из ассортимента для озеленения Москвы (документ заказчика) — только виды с «+» для типа территории участка, без инвазивных (369-ПП), с учётом кодов ограничений ассортимента и норм по породе; для дворов в приоритете базовый ассортимент 515-ПП. Основание подбора видно на панели GreenPlan и в записке.

Почему на участке один стиль, а не лучший приём для каждой зоны? Проектировщик сначала выбирает замысел участка, потом решает зоны в нём. Если выбирать приём для каждой зоны отдельно, в одном дворе оказываются волны рядом с диагоналями. GreenPlan сначала выбирает стиль участка и ведущий аналог, потом приёмы зон в этом стиле, и сажает одну палитру видов на весь участок.

Как проверяется, что паттерн реально подходит участку? Retrieval — по числовым признакам (площадь, доля пригодной площади, состав зон), не текстом; назначение паттерна — голосование к ближайших проектов с решением для этого вида зоны (слабые аналогии со сходством ниже 0,2 не голосуют, такие зоны помечены «проверить»), а не один "самый похожий" проект; итоговая расстановка проверяется независимым модулем нарушений (`violation_report.py`), а не тем же кодом, что её строил.

Почему в корпусе синтетические проекты? 14 реальных проектов — это то, что реально задокументировано (реальный DXF + реальная пояснительная записка). Среди них нет многих типичных городских случаев (круглая площадь, бульвар, двор с детской площадкой, парковка, сквер над сетями), и новому такому участку не с чем было бы сравниваться. 20 синтетических эталонов закрывают эти случаи законченными образцами без нарушений норм; каждый явно помечен как синтетический.

13. Другая документация проекта

- `../README.md` — быстрый старт (Docker/локально), список эндпоинтов, статус GreenPlan по этапам.
- `../backend/README.md` — `/api/generate-greenery` подробно: параметры, алгоритм, таблица отступов.
- `../data/norms/README.md` — какие нормативные акты извлечены и сверены с кодом, что ещё нет.
- `../observability/README.md` — метрики Prometheus, логи, дашборды Grafana.

- `../frontend/public/models/README.md` — 3D-модели, как подключить свой пак.
- `locations//design_rationale.md` — по каждому из 34 проектов корпуса: что реально в DXF и почему выбран паттерн озеленения.
- `CASE_BRIEF.md` — текст ТЗ кейса.
- `SOLUTION.md` — документация решения по ТЗ.
- `GREENPLAN_ALGORITHM.md` — алгоритм GreenPlan по шагам.
- `../examples/` — выходные DXF и файлы объяснений для трёх участков.